Tutorial

Reinforcement Learning

Từ nền tảng David Silver đến deep RL và nghiên cứu hiện đại

Ghi chú học tập cho nghiên cứu RL · 12 tháng 10 năm 2026 · v2.2 · Bài giảng tổng hợp

Mục tiêu của tài liệu

Hiểu formulation, công thức, cơ chế cập nhật và pipeline của RL để đọc nghiên cứu độc lập. Phần thực hành dừng ở phân tích dữ liệu, target, module học và protocol đánh giá; không yêu cầu viết code. Tài liệu này là bản diễn giải có chọn lọc, không thay thế toàn bộ bốn nguồn gốc.

Trục kiến thức. David Silver, Lectures 1–9, quyết định trình tự và trực giác chính. Sutton–Barto, Chương 2–13, bổ sung định nghĩa, điều kiện và các bước suy luận. Berkeley CS185/285 **Spring 2026** mở rộng sang deep RL, imitation, model-based/world models, offline RL, inference, multi-task/hierarchy và LLM RL. Boyd–Vandenberghe nằm ở phần bổ trợ cuối, chỉ giữ tối ưu hóa cần cho đọc phương pháp.[1, 2, 3, 4]

Cách đọc. Đọc Mục 1–6 trước; sau đó Mục 7–11 để nối với Silver 6–9. Mục 12–18 là phần mở rộng Berkeley. Có thể đọc Mục 19 về tối ưu hóa khi gặp gradient, KL constraint hoặc Lagrangian. Mỗi mục dùng khung bảy phần của `RL_Lecture_Template.tex`. Sau công thức là cách đọc, ý nghĩa từng thành phần hoặc câu hỏi dẫn dắt; ví dụ ở phần tiếp theo giúp thực hiện phép tính. Kiến thức cũ được liên kết để đọc lại khi cần.

Mức độ và nguồn. Không gom mọi thuật toán thành một loss. Objective mô tả bài toán; update/loss mô tả cách học; metric đo đầu ra. Điều kiện của kết quả tabular được nêu riêng, không suy rộng sang neural networks. Công thức được đối chiếu với các nguồn trong Phụ lục B; ký hiệu và điều kiện áp dụng được giải thích trong bài; các ví dụ số và câu hỏi tự luyện là **minh họa tự dựng**, không phải kết quả thực nghiệm của nguồn.

Nguồn	Vai trò và phần sử dụng
Silver, UCL 2015	Lectures 1–5: MDP, Bellman, DP, MC/TD, control; 6–9: approximation, policy gradient, planning, exploration.
Sutton–Barto, 2e	Ch. 2–8: bandits và tabular methods; Ch. 9–13: approximation, off-policy, traces và policy gradient. PDF tác giả, bản hiệu đính 2020.
Berkeley, Spring 2026	Lectures 2–3, 4–20, 23–25; chọn nội dung bổ sung trực tiếp cho trục nền tảng. Không tổng hợp homework/API.
Boyd–Vandenberghe	Ch. 2–3; §§4.1–4.2, 4.7; Ch. 5; §§9.1–9.3. Không triển khai toàn bộ các họ solver.

Mục lục

Quy ước ký hiệu dùng xuyên suốt 4

I Nền tảng RL và phương pháp tabular 5

1 RL, MDP và return 5

2 Value functions và phương trình Bellman 7

3 Dynamic programming và generalized policy iteration 9

4 Monte Carlo và temporal-difference prediction 11

5 Sarsa, Expected Sarsa và Q-learning 13

6 Multi-step returns, eligibility traces và off-policy estimation 15

II Xấp xỉ hàm và tối ưu policy 17

7 Function approximation và Deep Q-Networks 17

8 Policy gradient, REINFORCE và actor–critic 19

9 Policy updates hiện đại: TRPO, PPO và maximum-entropy RL 21

III Khám phá, planning và world models 23

10 Exploration, bandits và thu thập thông tin 23

11 Learning, planning, Dyna và model-based RL 25

12 World models, latent states và partial observability 27

IV Học từ dữ liệu và suy luận 29

13 Imitation learning và inverse RL 29

14 Offline RL, data support và offline-to-online 31

15 Variational inference và control as inference 33

V Khái quát hóa và nghiên cứu hiện đại 35

16 Multi-task, goal-conditioned và hierarchical RL 35

17 RL với sequences và LLMs **37**

18 Lý thuyết, giới hạn và cách đọc thực nghiệm RL **39**

VI Nền tối ưu hóa bổ trợ **41**

19 Bổ trợ: những gì cần biết từ Convex Optimization **41**

VII Tự luyện và tra cứu **44**

Phụ lục A. Đáp án tự luyện **44**

Phụ lục B. Bản đồ nguồn để kiểm tra và học tiếp **47**

Quy ước ký hiệu dùng xuyên suốt

Thời gian và reward. Ở bước t, agent quan sát S_t (hoặc O_t), chọn A_t, rồi nhận R_{t+1} và S_{t+1}. Đây là cùng quy ước của Silver và Sutton–Barto. Trong Berkeley, $r(s_t, a_t)$ thường đánh chỉ số theo hành động; khi đổi sang ký hiệu ở đây, nó tương ứng reward sinh ra bởi chuyển tiếp (S_t, A_t), tức R_{t+1}. Không dịch chuyển reward thêm một bước khi tính return.

Ký hiệu	Ý nghĩa và phạm vi
$\mathcal{S}, \mathcal{A}$	Tập state và action. Công thức tổng hữu hạn dùng cho MDP hữu hạn; trường hợp liên tục cần tích phân/mật độ tương ứng.
S_t, A_t, R_{t+1}	Các biến ngẫu nhiên; s, a, r là giá trị cụ thể. O_t là observation; H_t là lịch sử quan sát, hành động và reward.
$\mathcal{P}^a_{ss'}$	$\Pr(S_{t+1} = s' \mid S_t = s, A_t = a)$, đúng ký hiệu model của Silver.
$\mathcal{R}^a_s$	$\mathbb{E}[R_{t+1} \mid S_t = s, A_t = a]$, expected immediate reward, không phải return.
γ, G_t, T	Discount $0 \le \gamma \le 1$; return từ t; T là thời điểm terminal của episode.
$\pi(a \mid s), \pi_\theta(a \mid s)$	Policy và policy có tham số θ. Behavior policy viết μ như Silver; Sutton–Barto dùng b cho cùng vai trò.
$v_\pi(s), q_\pi(s, a)$	Value thật của policy π. v_*, q_* là value tối ưu. Đây là chữ thường, khác các ước lượng bên dưới.
$V(s), Q(s, a)$	Các bảng ước lượng đang học, theo Silver. Chỉ số k chỉ vòng cập nhật DP; chỉ số t chỉ thời gian interaction.
$\hat{v}(s, w), \hat{q}(s, a, w)$	Value approximation với tham số critic w, theo Sutton–Barto. w^- là tham số target network đã giữ cố định.
$\alpha, \delta_t, \lambda$	Step size; TD error; hệ số trace/GAE. λ ở đây là ký hiệu thuật toán chuẩn, không phải objective tự đề xuất.
$\mathcal{D}, y, \hat{A}_t$	Dataset/replay buffer; target học; advantage ước lượng. A_t không có dấu mũ luôn là *action*.
$J(\theta), L(w)$	Performance objective và loss học. Tên L có thể dùng cho surrogate actor khi tham số là θ, được nêu tại chỗ.

Những ký hiệu chỉ dùng ở phần mở rộng. ρ_t là importance ratio; η là tham số learned dynamics; z_t là latent state; g là goal. Đây là các ký hiệu chuẩn của các phần tương ứng, được định nghĩa khi xuất hiện. Với optimization, x, f_i, ν giữ ký hiệu Boyd; multiplier λ_i có chỉ số để phân biệt với trace λ. Berkeley thường dùng ϕ cho critic; ở đây thống nhất thành w. Không đổi ý nghĩa toán học của công thức khi đổi tên biến.

Objective mặc định. Trừ khi một mục nói khác, tài liệu xét expected discounted return, initial-state distribution cố định và độc lập với θ, reward/dynamics không phụ thuộc trực tiếp vào tham số policy. Episode hữu hạn dùng $v(S_T) = 0$; continuing task mặc định $\gamma < 1$ và reward bị chặn. Các bài average-reward trong Ch. 10, 13 Sutton–Barto được phân biệt khái quát, không trộn vào công thức discounted.

I. Nền tảng RL và phương pháp tabular

1 RL, MDP và return

Nền tảng: Silver 1–2; Sutton–Barto Ch. 1, §§3.1–3.4.

> **Mục tiêu học**
> Mô tả một task bằng state, action, reward và horizon; phân biệt observation với state; tính return và xác định objective.

1. Hiểu nhanh

RL học cách ra quyết định khi hành động ảnh hưởng cả reward hiện tại lẫn dữ liệu tương lai. Một hành động ít reward trước mắt có thể mở đường tới kết quả tốt hơn. Policy quyết định hành vi; value dự đoán kết quả dài hạn; model dự đoán chuyển tiếp. Agent không bắt buộc phải có đủ cả ba thành phần.[1, Lectures 1–2]

Cần biết trước. Xác suất có điều kiện, kỳ vọng và tổng hình học. Khi reward ngẫu nhiên, objective dùng kỳ vọng; một rollout tốt riêng lẻ chưa xác định chất lượng policy.

2. Bài toán được giải quyết

Setting. MDP hữu hạn trong Silver gồm $(\mathcal{S}, \mathcal{A}, \mathcal{P}, \mathcal{R}, \gamma)$. Điều kiện Markov có nghĩa phân phối chuyển tiếp và reward, sau khi biết state và action hiện tại, không cần toàn bộ lịch sử. Một ảnh camera đơn lẻ có thể thiếu vận tốc hoặc vật bị che khuất, nên không tự động là state Markov.

Input/output. Khi học online: các chuyển tiếp $(S_t, A_t, R_{t+1}, S_{t+1})$; output thường là policy hoặc value để suy ra policy. Khi dùng: state/observation/history hiện tại đi tới action. Nếu chỉ có dataset có sẵn thì chuyển sang setting offline ở Mục 14.

Objective và metric. Objective là maximize expected return. Metric có thể là evaluation return hoặc success; success không tự động bằng reward dùng khi train.

3. Ý tưởng và công thức cốt lõi

$$G_t = \sum_{k=0}^{T-t-1} \gamma^k R_{t+k+1}. \tag{1}$$

Từ thời điểm t trở đi, ta thực sự nhận được gì? G_t cộng tất cả rewards còn lại của episode, nhưng reward càng xa càng được nhân với nhiều lần γ. Hạng đầu là R_{t+1}, hạng sau là γR_{t+2}; vì thế k đếm khoảng cách từ hiện tại, không phải thời gian tuyệt đối. $T-t$ là số bước còn lại. Với continuing task, tổng kéo dài tới vô hạn; reward bị chặn và $\gamma < 1$ giúp tổng hữu hạn.

γ gần 1 giữ trọng lượng của rewards xa tương đối lớn; γ nhỏ làm chúng ảnh hưởng ít hơn. Task hữu hạn có thể chọn $\gamma = 1$. Discount áp theo thời gian, không phải hệ số riêng cho mỗi loại outcome.

$$J(\theta) = \mathbb{E}_{\pi_\theta}[G_0]. \tag{2}$$

Một lần chạy cho một G_0; chạy lại có thể cho kết quả khác. $J(\theta)$ lấy trung bình lý thuyết của những kết quả đó khi dùng policy π_θ. Kỳ vọng bao gồm state ban đầu, action được sample và sự ngẫu nhiên của environment. **Ta tối ưu điều gì?** Chọn tham số θ để J lớn hơn, chứ không chỉ làm một trajectory riêng lẻ thật tốt. Trung bình return trên nhiều rollouts là một cách ước lượng J.

$$G_t = R_{t+1} + \gamma G_{t+1}. \tag{3}$$

Hãy tách chuỗi rewards thành **bước vừa tới** và **phần còn lại**. R_{t+1} là bước vừa tới; G_{t+1} đã cộng phần còn lại từ thời điểm kế tiếp. Nhân phần đó với γ để lùi toàn bộ về một bước. Ví dụ, nếu reward kế tiếp là 1, return sau đó là 4 và $\gamma = 0.9$, thì $G_t = 4.6$. Ở terminal, $G_T = 0$. Đây là đẳng thức về return, chưa phải lệnh cập nhật value.

4. Ví dụ và pipeline

Reward của một episode ba bước là $1, 0, 4$, $\gamma = 0.9$. Khi đó $G_0 = 1 + 0.9 \cdot 0 + 0.9^2 \cdot 4 = 4.24$; $G_1 = 3.6$; $G_2 = 4$. Nếu có action kết thúc ngay với reward 3, nó tốt hơn ở bước đầu theo immediate reward nhưng kém hơn theo return trong ví dụ này.

1. **Dữ liệu:** Agent chọn action, environment sinh next observation và reward.
2. **Học:** Thu transition/trajectory; thuật toán sẽ tạo target và cập nhật policy/value/-model.
3. **Dùng và đánh giá:** Policy chọn action với thông tin thật sự có tại deployment; đo return trên các rollout đánh giá riêng.

5. So sánh, giới hạn và ứng dụng

Supervised learning thường nhận nhãn đích cho từng input; RL nhận tín hiệu đánh giá hậu quả, thường trễ. Planning dùng model để tính hành động; learning dùng dữ liệu để cải thiện thành phần chưa biết. Robot navigation là ví dụ minh họa: observation từ sensor, action là lệnh chuyển động, reward gắn với task, metric là success/collision/return. Reward được tối ưu có thể không phản ánh đầy đủ mục tiêu thực tế.

6. Tự luyện

1. Episode có rewards $2, -1, 3$, $\gamma = 0.5$: tính G_0, G_1.
2. Vì sao ảnh vị trí hiện tại có thể không đủ cho bài toán điều khiển chuyển động?
3. Nếu simulator dừng vì giới hạn chạy, đó có chắc là terminal của task không?

Đáp án: phụ lục tự luyện.

7. Điều cần nhớ và đọc thêm

Reward là feedback một bước; return là tổng theo thời gian; objective là kỳ vọng trên trajectory. State phải đủ cho quyết định. Đọc Silver 1–2 và Sutton–Barto §§3.1–3.4; chuyển tới value ở Mục 2.

2 Value functions và phương trình Bellman

Nền tảng: Silver Lecture 2; Sutton–Barto §§3.5–3.7.

> **Mục tiêu học**
> Tính v_π, q_π và giải thích vì sao Bellman liên kết giá trị hiện tại với tương lai; phân biệt evaluation với optimality.

1. Hiểu nhanh

Value không cho biết state đẹp hay xấu nói chung; nó cho biết expected return khi theo *một policy cụ thể*. q_π còn cố định action đầu tiên, sau đó tiếp tục theo π. Bellman biến một tổng reward dài thành quan hệ giữa các state lân cận, nhờ đẳng thức (3).[1, Lecture 2]

2. Bài toán được giải quyết

Prediction: policy đã cho, cần v_π/q_π. Control: cần policy có return lớn. Model có thể biết hoặc chưa biết; Bellman là quan hệ đúng của MDP, không đòi hỏi agent đã biết model để định nghĩa nó. Output value khác output action; muốn hành động phải thêm quy tắc chọn action.

3. Ý tưởng và công thức cốt lõi

$$v_\pi(s) = \mathbb{E}_\pi[G_t \mid S_t = s], \qquad q_\pi(s,a) = \mathbb{E}_\pi[G_t \mid S_t = s, A_t = a]. \tag{4}$$

Đứng ở s thì tốt đến đâu? $v_\pi(s)$ lấy expected return nếu từ đó làm theo π. **Nếu buộc chọn a trước thì sao?** $q_\pi(s,a)$ cố định action đầu, rồi mới làm theo π ở các bước sau. Ký hiệu sau dấu $\mid$ là thông tin được giữ cố định; những ngẫu nhiên còn lại vẫn được lấy kỳ vọng. Cả hai phụ thuộc policy: cùng state có thể có value khác dưới hai policies.

$$v_\pi(s) = \mathbb{E}_\pi[R_{t+1} + \gamma v_\pi(S_{t+1}) \mid S_t = s]. \tag{5}$$

Đọc vế phải là: **reward bước tới + giá trị kỳ vọng của tương lai, có discount**. Sau khi tới S_{t+1}, ta vẫn tiếp tục theo π, nên dùng $v_\pi(S_{t+1})$. Dấu kỳ vọng trung bình cả action mà policy chọn lẫn chuyển tiếp/reward của environment. Như vậy không cần liệt kê mọi trajectory dài để mô tả value: chỉ cần nối value hiện tại với next-state value. Một transition đơn lẻ là một sample của vế phải, không phải chính kỳ vọng đó.

$$q_\pi(s,a) = \mathcal{R}_s^a + \gamma \sum_{s'} \mathcal{P}_{ss'}^a v_\pi(s'). \tag{6}$$

Action đầu đã cố định là a, nên reward trung bình là $\mathcal{R}_s^a$. Environment có thể đưa ta tới nhiều s': lấy $v_\pi(s')$ của từng nơi, nhân probability $\mathcal{P}_{ss'}^a$, rồi cộng lại. Cuối cùng nhân γ. **Vì sao cần tổng theo s'?** Ta chưa biết next state nào sẽ xảy ra. Terminal có value zero, nên nhánh đi tới terminal chỉ đóng góp immediate reward.

$$v_\pi(s) = \sum_a \pi(a \mid s) q_\pi(s,a). \tag{7}$$

Ở state s, policy có thể chọn nhiều actions. Mỗi action có expected return $q_\pi(s,a)$ và probability được chọn $\pi(a \mid s)$. Vì vậy $v_\pi(s)$ là **trung bình có trọng số** của các action-values. Nếu policy chọn một action chắc chắn, tổng chỉ còn value của action đó; nếu chọn đều hai actions, value là trung bình hai q-values. Đây là averaging theo policy hiện tại, không phải chọn action tốt nhất.

Bellman expectation giữ policy cố định. Bellman optimality thay việc lấy trung bình action theo policy bằng chọn action tốt nhất:

$$q_*(s,a) = \mathcal{R}_s^a + \gamma \sum_{s'} \mathcal{P}_{ss'}^a \max_{a'} q_*(s',a'). \tag{8}$$

Sau khi thực hiện a và tới s', công thức giả định ta chọn action tiếp theo tốt nhất theo q_*. Vì thế có $\max_{a'}$ bên trong tổng. **Vì sao không max theo next state?** Agent chọn action, không chọn được state mà environment ngẫu nhiên sinh ra; do đó vẫn phải lấy expectation theo $\mathcal{P}_{ss'}^a$. q_* mô tả kết quả tối ưu của MDP, khác một estimate Q đang học.

Ở finite discounted MDP, có stationary deterministic optimal policy. Điều này không nói rằng một neural policy optimizer sẽ tìm được policy đó. Cũng không áp nguyên phát biểu sang partial observability hoặc objective khác.

4. Ví dụ và pipeline

State s có action a cho reward 2, rồi tới u với xác suất 0.75, terminal với 0.25. Nếu $v_\pi(u) = 4$, $\gamma = 0.9$, thì $q_\pi(s,a) = 2 + 0.9(0.75 \cdot 4) = 4.7$. Action b kết thúc ngay với reward 3 nên $q_\pi(s,b) = 3$. Policy chọn đều hai action có $v_\pi(s) = 3.85$.

1. **Dữ liệu:** Biết model thì dùng expectation chính xác; chưa biết thì lấy sample transition.
2. **Học:** Ước lượng value thỏa quan hệ Bellman bằng backup hoặc learning update.
3. **Dùng và đánh giá:** Greedy với q_* đạt control tối ưu trong setting trên; greedy với Q đang học có thể sai.

5. So sánh, giới hạn và ứng dụng

q_π không bằng q_* trừ khi policy đủ tốt. Bellman error nhỏ trên dữ liệu đã thấy không kiểm tra tất cả state-action. Value hữu ích cho chọn đường, phân bổ hành động và làm critic; nó không phải xác suất success trừ khi reward/horizon được thiết kế để có ý nghĩa đó.

6. Tự luyện

1. Nếu ở ví dụ $v_\pi(u) = 2$ thì $q_\pi(s,a)$ và $v_\pi(s)$ là bao nhiêu?
2. Vì sao greedy với q_π có thể đổi policy và do đó đổi cả value?
3. Chỉ có v_π nhưng không có model: có luôn chọn greedy action được không?

Đáp án: phụ lục tự luyện.

7. Điều cần nhớ và đọc thêm

Bellman là recursive expectation, không phải mặc định TD loss. Dùng expectation để đánh giá policy, dùng max để mô tả optimality. Đọc Sutton–Barto §3.5 rồi chuyển tới DP.

3 Dynamic programming và generalized policy iteration

Nền tảng: Silver Lecture 3; Sutton–Barto Ch. 4.

> **Mục tiêu học**
> Thực hiện một Bellman backup; phân biệt policy evaluation, policy improvement và value iteration.

1. Hiểu nhanh

DP giải quyết MDP có model đã biết. Thay vì trải nghiệm từng đường đi, nó dùng model để tính expected return. Evaluation trả lời policy hiện tại tốt thế nào; improvement thay hành vi dựa trên value. Xen kẽ hai bước tạo policy iteration.[1, Lecture 3]

2. Bài toán được giải quyết

Input: finite MDP, model chuyển tiếp/reward; khi evaluation có thêm π. Output: value/policy. Objective vẫn là return ở Mục 1; không có supervised dataset hoặc neural-network loss bắt buộc. Metric có thể là value error nếu biết lời giải, hoặc return của policy suy ra.

3. Ý tưởng và công thức cốt lõi

$$V_{k+1}(s) = \sum_a \pi(a \mid s) \left[\mathcal{R}_s^a + \gamma \sum_{s'} \mathcal{P}_{ss'}^a V_k(s') \right]. \tag{9}$$

Đây là **một vòng tính lại value khi policy chưa đổi**. Với mỗi s, dùng V_k để đánh giá tương lai; lấy trung bình theo next states, rồi lấy trung bình theo actions của π. Kết quả ghi vào $V_{k+1}(s)$. Chỉ số k đếm vòng backup, không phải bước interaction. Công thức dùng model đã biết và mọi nhánh có thể xảy ra; nó không cần sample một trajectory thật cho mỗi backup.

$$\pi'(s) \in \arg\max_a q_\pi(s, a). \tag{10}$$

arg max trả về **action đạt giá trị lớn nhất**, còn max chỉ trả về con số value lớn nhất. $\pi'(s)$ là action của policy mới ở s: chọn action tốt nhất theo q_π của policy cũ. Nếu nhiều actions cùng đạt maximum, chọn một theo tie rule đã quy định. Ở đây π' được viết như deterministic policy; bước này đổi policy sau khi đã có action-values để so sánh.

Policy improvement theorem cho biết greedy cải thiện theo value của policy cũ trong các điều kiện MDP tương ứng. Nếu chỉ dùng value xấp xỉ chưa hội tụ, không được coi theorem này là bảo đảm mỗi update đều tốt hơn.

$$V_{k+1}(s) = \max_a \left[\mathcal{R}_s^a + \gamma \sum_{s'} \mathcal{P}_{ss'}^a V_k(s') \right]. \tag{11}$$

Khác policy evaluation, ở mỗi state ta không average theo một policy cố định mà chọn action có backup lớn nhất. Value ở vòng k cung cấp dự đoán tương lai; vòng $k+1$ kết hợp nó với reward/model để cải thiện estimate tối ưu. **Max có biết tương lai thật không?** Không: nó chỉ so sánh các dự đoán dựa trên V_k. Reward trễ được truyền ngược qua các vòng backup.

Vì sao hội tụ? Với $\gamma < 1$, Bellman operators là contraction trong max norm. Một backup làm khoảng cách giữa hai value vectors co lại tối đa hệ số γ. Repeated backups đưa tới fixed point duy nhất. Trường hợp $\gamma = 1$ cần assumptions episodic/proper tương ứng, không lấy contraction trên làm bằng chứng.

Generalized policy iteration (GPI). Evaluation và improvement có thể chưa hoàn tất trước khi chuyển bước; điều quan trọng là chúng cùng tác động tới value và policy. GPI là mô hình tư duy cho nhiều phương pháp ở phần sau, không phải một thuật toán chỉ có một update.[2, §4.6]

4. Ví dụ và pipeline

MDP deterministic: ở u, nhận 4 rồi terminal; ở s, action a nhận 0 rồi tới u, action b nhận 2 rồi terminal. $\gamma = 0.9$, $V_0 = 0$. Backup đồng bộ lần 1: $V_1(u) = 4, V_1(s) = 2$. Lần 2: $V_2(s) = \max(3.6, 2) = 3.6$. Reward trễ truyền lùi qua hai lần backup.

1. **Dữ liệu:** Đọc model và khởi tạo value.
2. **Học:** Backup theo policy hoặc max; policy iteration còn có bước greedy improvement.
3. **Dùng và đánh giá:** Trích policy từ value và model; evaluation action chọn từ model-based lookahead.

5. So sánh, giới hạn và ứng dụng

Policy iteration đánh giá policy rồi cải thiện; value iteration dùng max ngay trong backup. MC/TD ở mục sau thay expectation trên model bằng kinh nghiệm. DP tabular có thể đắt khi state/action lớn; đây là vấn đề biểu diễn và sweep, khác chi phí thu dữ liệu. Ví dụ ứng dụng minh họa là planning grid navigation có map và transition đã biết.

6. Tự luyện

1. Ở ví dụ, đổi $\gamma = 0.4$: policy greedy cuối chọn a hay b?
2. Nếu update u trước rồi dùng giá trị mới cho s ngay trong một sweep, kết quả lần 1 có khác backup đồng bộ không?
3. Model sai có được contraction bảo đảm policy đúng trong environment thật không?

Đáp án: phụ lục tự luyện.

7. Điều cần nhớ và đọc thêm

DP cần model; Bellman dùng model để backup; GPI liên kết prediction với control. Đọc Silver 3 và Sutton–Barto §§4.1–4.6.

4 Monte Carlo và temporal-difference prediction

Nền tảng: Silver Lecture 4; Sutton–Barto §§5.1–5.2, 6.1–6.3.

> **Mục tiêu học**
> Phân biệt sample return với bootstrap target; tính một update MC/TD và giải thích bias, variance.

1. Hiểu nhanh

MC đợi kết quả episode rồi dùng return thực tế làm target. TD dùng reward vừa nhận cộng value ước lượng của state kế tiếp. Cả hai học từ dữ liệu, không cần biết transition model; TD vẫn dùng một dự đoán học được ở vế target, gọi là bootstrapping.[1, Lecture 4]

2. Bài toán được giải quyết

Policy π cố định; input là trajectories/transitions từ policy đó, output là $V \approx v_\pi$. Đây là prediction, chưa phải control. Objective dự đoán value phải tách khỏi objective chọn policy của RL. Nếu có true value, đo value error; nếu không, Bellman residual/held-out target chỉ kiểm tra khía cạnh dự đoán, không thay task return.

3. Ý tưởng và công thức cốt lõi

$$V(S_t) \leftarrow V(S_t) + \alpha[G_t - V(S_t)]. \tag{12}$$

Đây là cách sửa estimate theo **sai lệch giữa kết quả đã thấy và dự đoán cũ**. G_t là return sau khi episode hoàn tất; $G_t - V(S_t)$ là prediction error. Nếu error dương, tăng V; âm thì giảm. α quyết định mức sửa, không thay reward của task. Với $0 < \alpha \leq 1$, update đi từ estimate cũ về phía target. Chọn $\alpha = 1/N(S_t)$ theo số mẫu được cập nhật cho cách lấy sample mean.

First-visit MC cập nhật lần đầu state xuất hiện trong một episode; every-visit dùng mỗi lần xuất hiện. Cả hai cần xác định episode và cách đếm mẫu. Return là sample của một biến ngẫu nhiên, không phải true value.

$$\delta_t = R_{t+1} + \gamma V(S_{t+1}) - V(S_t). \tag{13}$$

Chuyển tiếp vừa thấy tốt hơn hay kém hơn dự đoán? TD dựng target $R_{t+1} + \gamma V(S_{t+1})$, rồi trừ estimate hiện tại $V(S_t)$. Hiệu đó là δ_t. Target dùng reward thật nhưng dùng *dự đoán* cho phần tương lai: đây là bootstrapping. Nếu S_{t+1} terminal, đặt $V(S_{t+1}) = 0$. TD error dương không có nghĩa reward dương; nó có nghĩa target cao hơn dự đoán cũ.

$$V(S_t) \leftarrow V(S_t) + \alpha\delta_t. \tag{14}$$

Sau khi tính TD error, chỉ sửa estimate của state vừa thăm một lượng $\alpha\delta_t$. Ví dụ $\delta_t = 2$, $\alpha = 0.1$ thì tăng $V(S_t)$ thêm 0.2; nếu error là -2 thì giảm 0.2. Điều quan trọng là **có thể học ngay từ một transition**, trước khi episode kết thúc. Số value mới vẫn là estimate; một update không làm nó trở thành true value.

Lập luận. Bellman expectation nói true value bằng expected one-step target. TD dùng một realization của target đó, rồi đưa estimate hiện tại tới gần nó. Nếu next-state estimate sai, target có thể sai; nếu chờ toàn episode, MC tránh bootstrap nhưng chịu randomness từ cả đường đi.

Bias và variance. MC return là target không bias cho value của policy đang thu dữ liệu trong setting phù hợp; TD target chịu bias do value approximation hiện tại. MC thường có variance lớn vì nhiều bước ngẫu nhiên; TD thường giảm variance và học sớm hơn. Đây là xu hướng, không phải khẳng định mọi task TD tốt hơn MC.[2, §6.2]

4. Ví dụ và pipeline

Cho $V(s) = 1, V(u) = 2$, transition $s \to u$ reward 0, $\gamma = 0.9$, $\alpha = 0.5$. TD target là 1.8, nên $V(s)$ mới bằng 1.4. Nếu episode sau đó nhận reward 4 rồi kết thúc, MC return ở s là 3.6, cho $V(s)$ mới 2.3 từ cùng estimate ban đầu.

1. **Dữ liệu:** MC thu episode đầy đủ; TD có thể dùng từng transition.
2. **Học:** MC tạo G_t; TD tạo reward cộng discounted next-state value; cập nhật bảng V.
3. **Dùng và đánh giá:** Prediction giữ policy cố định; khi gắn vào control phải tính tới policy thay đổi.

5. So sánh, giới hạn và ứng dụng

DP dùng expectation và bootstrap; MC dùng sample và không bootstrap; TD dùng sample và bootstrap. Tabular TD(0) on-policy có bảo đảm hội tụ với visitation và step-size phù hợp. Khi policy đổi, target value đổi theo; khi function approximation/off-policy, điều kiện khác. Ví dụ prediction là ước lượng tổng chi phí tương lai của một controller đang vận hành.

6. Tự luyện

1. Với $V(s) = 2, V(u) = 5, R_{t+1} = 1, \gamma = 0.8, \alpha = 0.1$, tính TD target, δ_t và estimate mới.
2. Nếu next state terminal, target là gì?
3. Vì sao MC target và TD target có thể khác trên cùng transition?

Đáp án: phụ lục tự luyện.

7. Điều cần nhớ và đọc thêm

Không bootstrap không có nghĩa không có stochastic error. TD error không phải return. Đọc Silver 4 và Sutton–Barto §6.2; control ở Mục 5.

5 Sarsa, Expected Sarsa và Q-learning

Nền tảng: Silver Lecture 5; Sutton–Barto §§5.3–5.4, 6.4–6.7.

> **Mục tiêu học**
> Tính ba target và nhận ra target policy/behavior policy; nêu đúng phạm vi bảo đảm tabular Q-learning.

1. Hiểu nhanh

Control vừa học value vừa dùng value để thay policy. Sarsa đánh giá hành vi đang thực hiện, kể cả exploration. Q-learning backup theo action tốt nhất ở bước sau dù dữ liệu do một policy khác sinh ra. Khác biệt này nằm ở target, không chỉ ở cách chọn action hiện tại.[1, Lecture 5]

2. Bài toán được giải quyết

Input: transitions online hoặc dữ liệu có coverage phù hợp; output: bảng Q và policy suy ra. Behavior μ thu dữ liệu; target π là policy value muốn học. On-policy dùng cùng policy cho hai vai trò; off-policy cho phép khác nhau. Objective control là return; TD squared error nếu dùng để fit critic chỉ là phương tiện học.

3. Ý tưởng và công thức cốt lõi

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha[R_{t+1} + \gamma Q(S_{t+1}, A_{t+1}) - Q(S_t, A_t)]. \tag{15}$$

Trong ngoặc vuông là TD error cho pair (S_t, A_t). Target dùng $Q(S_{t+1}, A_{t+1})$, với A_{t+1} là action thật sự sample theo policy đang đánh giá. Do đó action exploratory ở bước sau cũng đi vào target. Tên Sarsa nhắc bộ dữ liệu $S_t, A_t, R_{t+1}, S_{t+1}, A_{t+1}$. Ở terminal không có future action-value để bootstrap; target còn reward. Update chỉ sửa pair hiện tại trong bảng.

$$y = R_{t+1} + \gamma \sum_a \pi(a \mid S_{t+1}) Q(S_{t+1}, a). \tag{16}$$

Thay vì chờ biết một A_{t+1} đã sample, ta average next-action values theo $\pi(a \mid S_{t+1})$. **Ngẫu nhiên nào được bỏ bớt?** Ngẫu nhiên do chọn next action được tính expectation; next state/reward ở transition vẫn là samples. y là target, chưa phải value mới: dùng $Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha[y - Q(S_t, A_t)]$. Terminal có $y = R_{t+1}$.

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha[R_{t+1} + \gamma \max_a Q(S_{t+1}, a) - Q(S_t, A_t)]. \tag{17}$$

Target hỏi: **ở next state, nếu từ đó chọn action có estimate lớn nhất thì kết quả là gì?** Vì dùng max, nó không cần action exploratory mà behavior sẽ thật sự chọn ở bước sau. Đây là điểm làm Q-learning off-policy. Max ở đây dùng Q đang học, nên có thể chọn action bị overestimate; nó chưa phải q_*. Ở terminal, bỏ phần bootstrap và chỉ dùng reward.

Vì sao Q-learning là off-policy? Sau transition thật, target giả định future action greedy với Q. Behavior vẫn có thể chọn action exploratory. Tabular one-step Q-learning không cần action importance ratio: sau khi điều kiện theo (s, a), sample next transition theo dynamics của environment. Điều này không biến dữ liệu thiếu coverage thành đủ, cũng không miễn mọi actor-critic khỏi correction.

Điều kiện hội tụ. Finite stationary MDP, reward bị chặn, $\gamma < 1$, mọi state-action được thăm vô hạn và step sizes riêng cho từng pair có tổng vô hạn nhưng tổng bình phương hữu hạn là bộ điều kiện chuẩn đủ cho tabular Q-learning. Với Sarsa control còn cần policy improvement/exploration phù hợp, ví dụ GLIE cùng điều kiện step size. Một lịch ϵ giảm không tự chứng minh đã thăm đủ mọi pair.[2, 1, §6.5; Lecture 5]

4. Ví dụ và pipeline

$Q(s,a) = 2$, $R = 1$, $\gamma = 0.9$, $\alpha = 0.5$. Next state có $Q(u,L) = 4, Q(u,R) = 0$, policy chọn L với xác suất 0.75, nhưng action sample thực tế là R. Sarsa target 1 nên estimate mới 1.5; Expected Sarsa target 3.7 nên mới 2.85; Q-learning target 4.6 nên mới 3.3.

1. **Dữ liệu:** Behavior chọn action, thường có exploration, thu chuyển tiếp.
2. **Học:** Tạo target bằng sampled action, expected policy action hoặc max; update đúng một pair ở tabular.
3. **Dùng và đánh giá:** Deployment có thể greedy; đánh giá phải ghi exploration có còn bật không và tie xử lý ra sao.

5. So sánh, giới hạn và ứng dụng

Sarsa có thể phản ánh risk do exploratory behavior; Q-learning học value cho greedy target. Không suy ra Sarsa luôn an toàn hoặc Q-learning luôn nguy hiểm: phụ thuộc task, reward và behavior. Max trên các estimate nhiễu có thể gây maximization bias; Double Q-learning tách action selection khỏi value evaluation, không bảo đảm xóa mọi estimation error.[2, §6.7]

6. Tự luyện

1. Giữ ví dụ nhưng $\pi(L \mid u) = 0.5$, $A_{t+1} = L$. Tính ba target và ba estimate mới.
2. ϵ-greedy cố định 0.1 có phải GLIE không?
3. Một action chưa từng có trong dataset nhưng có Q lớn: tabular theorem có bảo đảm estimate đó đúng không?

Đáp án: phụ lục tự luyện.

7. Điều cần nhớ và đọc thêm

Sarsa dùng sampled future action; Expected Sarsa lấy expectation; Q-learning dùng max. Off-policy là quan hệ giữa policies. Đọc Silver 5 và Sutton–Barto §§6.4–6.7.

6 Multi-step returns, eligibility traces và off-policy estimation

Nền tảng: Silver 4–5; Sutton–Barto Ch. 7, Ch. 12 và §5.5.

Mục tiêu học

Giải thích credit assignment theo nhiều bước; tính n-step target và importance ratio; hiểu phạm vi tương đương forward/backward view.

1. Hiểu nhanh

One-step TD truyền reward xa qua nhiều lần cập nhật. Multi-step target dùng nhiều reward quan sát trực tiếp trước khi bootstrap. Eligibility traces lưu mức liên quan của những state/parameters đã hoạt động gần đây để một TD error có thể cập nhật cả quá khứ. Importance sampling xử lý một vấn đề khác: dữ liệu đến từ policy khác target.[1, Lecture 4]

2. Bài toán được giải quyết

Input: sequences thay vì transition rời rạc. Output: value hoặc action-value. Các trace formulas dưới đây dùng tabular on-policy prediction; không phải generic off-policy neural update. Khi behavior và target khác, phải xem estimator cụ thể có cần correction, có coverage hay có variance lớn không.

3. Ý tưởng và công thức cốt lõi

$$G_t^{(n)} = \sum_{k=0}^{n-1} \gamma^k R_{t+k+1} + \gamma^n V(S_{t+n}). \tag{18}$$

Ta nhìn thật n bước: cộng n rewards quan sát được, rồi nối thêm estimate tại S_{t+n} cho phần chưa quan sát. Vì estimate đó nằm cách hiện tại n bước nên hệ số là γ^n. $n = 1$ cho one-step TD target. Công thức viết cho $t + n < T$; nếu chạm terminal sớm hơn, dừng tổng tại terminal và bỏ bootstrap. V là phiên bản estimate dùng để tạo target, không tự thay giữa các hạng.

$$E_t(s) = \gamma\lambda E_{t-1}(s) + \mathbf{1}\{S_t = s\}. \tag{19}$$

Có thể hình dung $E_t(s)$ là **dấu vết của việc state s vừa được thăm**. Dấu cũ giảm bởi $\gamma\lambda$; indicator cộng 1 nếu state hiện tại đúng là s, còn lại cộng 0. Thăm lại làm accumulating trace tăng thêm, nên nó không phải probability và có thể lớn hơn 1. Khởi tạo $E_{-1}(s) = 0$ ở đầu episode. λ điều khiển độ lưu giữ credit, không phải một penalty trong objective.

$$V(s) \leftarrow V(s) + \alpha\delta_t E_t(s). \tag{20}$$

Cùng TD error hiện tại được gửi tới **mọi state còn trace**, nhưng state có trace lớn nhận update lớn hơn. Nhờ vậy reward mới có thể sửa các estimates của states đã đi qua gần đây, thay vì chỉ sửa state hiện tại. Với $\lambda = 0$, trace chỉ giữ lượt thăm hiện tại, trở về TD(0). Đây là tabular on-policy trace update; không tự gắn nguyên công thức vào một neural/off-policy method bất kỳ.

$\lambda = 0$ chỉ giữ state hiện tại, trả về TD(0). Forward view là trung bình có trọng số các n-step returns; backward view phân phối TD error qua traces. Tương đương cổ điển cần điều kiện offline/frozen estimates phù hợp. Online accumulating TD(λ) không được

coi đồng nhất mọi bước với forward view; true online TD(λ) giải quyết khác biệt này.[2, §§12.1–12.5]

$$\rho_t = \frac{\pi(A_t \mid S_t)}{\mu(A_t \mid S_t)}. \tag{21}$$

Tử số hỏi target policy muốn chọn action này thường đến đâu; mẫu số hỏi behavior đã chọn nó thường đến đâu. $\rho_t > 1$ làm mẫu này được nhấn mạnh khi đổi expectation từ behavior sang target; $\rho_t < 1$ làm nó bớt quan trọng. Ví dụ probabilities $0.8/0.4$ cho ratio 2. Điều kiện support rất quan trọng: nếu target cần một action mà behavior probability bằng zero, phép chia không tạo được thông tin còn thiếu.

Ratios theo nhiều bước được nhân để đổi trajectory distribution. Tích có thể rất lớn hoặc bằng zero; ordinary importance sampling có thể có variance cao. Weighted importance sampling có bias hữu hạn mẫu nhưng có thể giảm variance. Không lấy một ratio đơn lẻ rồi tự gắn vào bất kỳ loss nào.

4. Ví dụ và pipeline

Hai rewards tiếp theo $1, 2$, $\gamma = 0.9$, $V(S_{t+2}) = 3$: $G_t^{(2)} = 1 + 0.9 \cdot 2 + 0.9^2 \cdot 3 = 5.23$. Với $\gamma\lambda = 0.8$, một state chỉ được thăm ở bước đầu có trace $1, 0.8, 0.64$ ở ba bước liên tiếp. Nếu action có xác suất 0.8 dưới target và 0.4 dưới behavior, ratio là 2.

1. **Dữ liệu:** Lưu đoạn sequence và terminal boundary; ghi behavior probabilities nếu estimator cần.
2. **Học:** Tạo n-step target hoặc duy trì trace; correction off-policy phải theo thuật toán cụ thể.
3. **Dùng và đánh giá:** Đánh giá prediction/control riêng; kiểm tra data support và độ lớn của importance weights.

5. So sánh, giới hạn và ứng dụng

Multi-step thay bias/variance và tốc độ truyền credit; trace không giải quyết reward misspecification. Off-policy sampling không tạo ra thông tin về action có probability zero trong behavior. Các ý tưởng này nối sang GAE ở Mục 8 và offline RL ở Mục 14.

6. Tự luyện

1. Nếu episode kết thúc sau đúng hai rewards của ví dụ, target là bao nhiêu?
2. Tính trace bước sau nếu state vừa được thăm lại và trace trước là 0.64, $\gamma\lambda = 0.8$.
3. Target cần action có probability 0.2, behavior probability zero: dùng importance sampling được không?

Đáp án: phụ lục tự luyện.

7. Điều cần nhớ và đọc thêm

n-step và traces xử lý credit theo thời gian; importance sampling xử lý khác distribution. Không trộn hai vai trò. Đọc Sutton–Barto Ch. 7, §§12.1–12.5 và §5.5.

II. Xấp xỉ hàm và tối ưu policy

7 Function approximation và Deep Q-Networks

Nền tảng: Silver 6; Sutton–Barto Ch. 9–11. Mở rộng: Berkeley 4, 7–8.

> **Mục tiêu học**
> Phân biệt true value, bảng estimate và neural estimate; hiểu semi-gradient, replay, target network và deadly triad.

1. Hiểu nhanh

Khi state rất nhiều hoặc liên tục, không thể lưu một số riêng cho mọi state-action. Function approximation dùng cùng bộ tham số để chia sẻ thông tin giữa các inputs. Điều này giúp generalization nhưng cũng làm update ở một state thay đổi estimate ở state khác. DQN giữ cơ chế Q-learning, thay bảng Q bằng mạng $\hat{q}$ và thêm cách tổ chức dữ liệu/target.[1, Lecture 6]

2. Bài toán được giải quyết

Input: state hoặc state representation và transition data; output: $\hat{v}/\hat{q}$ với tham số w. DQN thường dùng action rời rạc, vì cần so sánh các action qua max. Deployment nhận state và chọn argmax; replay buffer và target network phục vụ training, không bắt buộc để chọn action.

3. Ý tưởng và công thức cốt lõi

Linear approximation. Với feature vector $x(s)$ cố định, $\hat{v}(s,w) = x(s)^\top w$. Neural approximation học cả representation và readout. Quality phụ thuộc feature, data distribution và target, không chỉ capacity.

$$w \leftarrow w + \alpha[y - \hat{v}(S_t, w)]\nabla_w \hat{v}(S_t, w). \tag{22}$$

$y - \hat{v}$ cho biết prediction cần tăng hay giảm. $\nabla_w \hat{v}$ cho biết sửa tham số theo hướng nào để thay prediction tại state đó. Nhân hai phần rồi dùng step size α. MC có $y = G_t$; TD có $y = R_{t+1} + \gamma\hat{v}(S_{t+1}, w)$. **Vì sao gọi semi-gradient?** Với TD, target cũng chứa w, nhưng update giữ target cố định khi lấy derivative: chỉ đi qua prediction hiện tại.

Vì bỏ derivative qua TD target, semi-gradient không phải gradient đầy đủ của squared Bellman residual. Trong trường hợp tuyến tính on-policy, nghiệm cố định thường là projected Bellman solution: quan hệ Bellman được khớp trong lớp hàm có thể biểu diễn. Nó không mặc định là nghiệm minimum true-value MSE.[2, Ch. 9, §11.4]

DQN target và loss.

$$y = R_{t+1} + \gamma(1-d)\max_a \hat{q}(S_{t+1}, a, w^-). \tag{23}$$

y là kết quả dài hạn mà mạng Q được yêu cầu dự đoán cho transition này. Reward là dữ liệu thật; next-state max lấy từ target network có tham số w^-. $d = 1$ nếu task terminal, làm $(1-d) = 0$; $d = 0$ thì giữ bootstrap. Vì vậy một kết thúc thật có target chỉ bằng reward. Simulator cutoff phải được phân biệt với terminal của task, nếu sau cutoff task về nguyên tắc vẫn tiếp tục.

$$L(w) = \mathbb{E}_{\mathcal{D}}[(y - \hat{q}(S_t, A_t, w))^2]. \tag{24}$$

So prediction $\hat{q}(S_t, A_t, w)$ của mạng đang học với target y, bình phương độ lệch, rồi average theo data/replay $\mathcal{D}$. Optimizer minimize $L(w)$ để fit targets đó. Trong bước gradient này, target được giữ cố định; w^- không được cập nhật cùng nhánh prediction. **Loss nhỏ có nghĩa gì?** Mạng khớp targets đã dùng tốt hơn; chưa chứng minh targets đúng, data đủ coverage hay policy có return cao.

Target network được giữ cố định giữa các lần refresh để target không thay quá nhanh. Replay lưu dữ liệu cũ và sample minibatches để dùng lại experience, giảm phụ thuộc vào thứ tự các transitions liền nhau; không làm toàn bộ data thật sự độc lập. Hai cơ chế hỗ trợ học, không phải chứng minh convergence cho deep Q-learning.

Deadly triad. Function approximation, bootstrapping và off-policy learning có thể kết hợp gây bất ổn/divergence. Ch. 11 Sutton–Barto phân tích vì sao bảo đảm tabular hoặc on-policy linear không thể chuyển thẳng sang off-policy neural setting. Gradient-TD, emphatic methods và residual-gradient có mục tiêu/assumptions riêng; chúng không đồng nghĩa thêm một target network.[2, §§11.2–11.9]

Double DQN. Mạng đang học chọn argmax ở next state; target network đánh giá action đó. Nó giảm nguồn bias do dùng cùng estimator để chọn và đánh giá action, không xóa mọi overestimation hay distribution shift.[3, Lecture 8]

4. Ví dụ và pipeline

Linear value có $x(s) = (1, 2)^\top, w = (0, 1)^\top$, target $y = 3$, $\alpha = 0.1$. Prediction là 2, gradient là $x(s)$; update thành $w = (0.1, 1.2)^\top$, prediction mới 2.5. Những state có features tương tự cũng đổi prediction.

1. **Dữ liệu:** DQN dùng exploratory policy thu transition, lưu replay, sample minibatches.
2. **Học:** Tạo target với w^-, update w bằng gradient loss; định kỳ refresh target.
3. **Dùng và đánh giá:** Giữ mạng cố định để đánh giá, chọn action từ $\hat{q}$; theo dõi return và value diagnostics.

5. So sánh, giới hạn và ứng dụng

Tabular update tác động một ô; function approximation có shared parameters. Với continuous action, exact max thường khó, nên actor-critic ở mục sau đưa một actor vào để đề xuất action. Minh họa application: policy điều khiển với state vector; với pixels, representation và observability phải được kiểm tra. Loss giảm chưa chứng minh hành động tốt hơn.

6. Tự luyện

1. DQN có reward 2, $\gamma = 0.9$, next target Q-values $(3, 5)$: tính y khi $d = 0$ và $d = 1$.
2. Ở ví dụ tuyến tính, nếu $y = 1$ thì tham số mới là gì?
3. Vì sao không áp theorem tabular Q-learning trực tiếp cho DQN?

Đáp án: phụ lục tự luyện.

7. Điều cần nhớ và đọc thêm

Value approximation thay cách biểu diễn; target xác định cái được học; optimizer chỉ fit theo target. Đọc Silver 6, Sutton–Barto §§9.3–9.5, §11.3 và Berkeley 8.

8 Policy gradient, REINFORCE và actor–critic

Nền tảng: Silver 7; Sutton–Barto Ch. 13. Mở rộng: Berkeley 5–6.

> **Mục tiêu học**
> Derive ý nghĩa score-function gradient; phân biệt return, baseline và advantage; hiểu actor/critic và GAE.

1. Hiểu nhanh

Policy gradient học trực tiếp cách chọn action bằng policy có tham số. Nó dùng log-policy gradient và kết quả trajectory, không cần đạo hàm qua environment. Actor–critic học thêm value để hỗ trợ tín hiệu cập nhật; critic sai có thể làm actor học sai.[1, Lecture 7]

2. Bài toán được giải quyết

Trajectories từ π_θ dùng để học actor θ và thường cả critic w; khi dùng có thể chỉ cần actor. Objective là Eq. (2). Công thức xét episodic discounted task, reward/dynamics độc lập θ và đủ regularity để đổi đạo hàm với kỳ vọng. Continuous action dùng policy density tương ứng.

3. Ý tưởng và công thức cốt lõi

Policy gradient.

$$\nabla_\theta J = \mathbb{E}_{\pi_\theta}\left[\sum_{t=0}^{T-1} \gamma^t \nabla_\theta \log \pi_\theta(A_t \mid S_t) G_t\right]. \tag{25}$$

$\nabla_\theta \log \pi_\theta(A_t \mid S_t)$ chỉ hướng tham số làm sampled action dễ xảy ra hơn; G_t cung cấp tín hiệu về hậu quả từ action đó trở đi. Nhân γ^t để phù hợp objective discounted bắt đầu từ thời điểm 0, rồi cộng các bước và lấy expectation. Đây là **expected gradient**; trên rollouts ta chỉ có estimator nhiễu của nó. Reward-to-go bỏ rewards xảy ra trước action, vì action hiện tại không gây ra quá khứ.

Không cần lấy derivative qua environment: policy probabilities cung cấp hướng cập nhật, còn rewards quan sát được cung cấp tín hiệu đánh giá. Tuy nhiên một rollout chịu ngẫu nhiên từ states, actions và tương lai; averaging nhiều rollouts giúp ước lượng expected gradient chứ không biến từng sample thành true gradient.

Baseline và advantage. Baseline phụ thuộc state nhưng không phụ thuộc action hiện tại có thể được trừ khỏi return mà không đổi expected score-function gradient, vì tổng gradient của action probabilities bằng zero. Tham số baseline phải được giữ cố định trong nhánh actor gradient của estimator này.

$$A_\pi(s,a) = q_\pi(s,a) - v_\pi(s). \tag{26}$$

Action a tốt hơn cách hành động thông thường của π ở state này bao nhiêu? $q_\pi(s,a)$ đánh giá action cụ thể; $v_\pi(s)$ đánh giá trung bình actions mà policy sẽ chọn. Lấy hiệu để có $A_\pi(s,a)$. Nếu dương, action tốt hơn mức trung bình ấy; nếu âm, kém hơn. Advantage âm không đồng nghĩa action luôn xấu: nó chỉ kém hơn baseline tại state này. A_π là advantage, còn A_t vẫn là action.

$$\theta \leftarrow \theta + \alpha\gamma^t[G_t - \hat{v}(S_t, w)]\nabla_\theta \log \pi_\theta(A_t \mid S_t). \tag{27}$$

Đọc update theo ba bước: quan sát return G_t; trừ dự đoán $\hat{v}(S_t, w)$ để biết kết quả tốt hơn hay kém hơn mức chờ đợi; dùng hiệu đó nhân với log-policy gradient. Hiệu dương cổ vũ sampled action, hiệu âm đi ngược hướng cổ vũ. α là mức cập nhật, γ^t là trọng số theo thời gian. Trong nhánh actor, giữ baseline cố định; critic w được học riêng từ return targets. Đây là một sample update, không bảo đảm mỗi bước đều tăng true J.

Actor–critic. One-step TD error từ critic có thể ước lượng advantage để cập nhật sớm, nhưng chịu critic error. Với shared network, tách gradients từng loss và giữ advantage cố định trong actor gradient.

GAE. Với $\delta_t = R_{t+1} + \gamma\hat{v}(S_{t+1}, w) - \hat{v}(S_t, w)$ ở đoạn rollout phù hợp:

$$\hat{A}_t^{\text{GAE}} = \sum_{l=0}^{T-t-1} (\gamma\lambda)^l \delta_{t+l}. \tag{28}$$

GAE gộp các **sai lệch TD hiện tại và tương lai** để đánh giá action tại t. Hạng gần được giữ mạnh hơn, hạng cách l bước nhân $(\gamma\lambda)^l$. $\lambda = 0$ chỉ dùng δ_t; $\lambda = 1$ trên episode đầy đủ cho $G_t - \hat{v}(S_t, w)$ nhờ các value terms triệt tiêu. Critic phải là snapshot dùng nhất quán cho đoạn tính này. Task terminal có bootstrap zero; rollout bị cắt giữa task vẫn cần next-value bootstrap phù hợp. Bootstrap với critic sai có thể tạo bias.

GAE thay bias/variance qua λ; baseline cố định với full MC return không tạo cùng nguồn bootstrap bias.

4. Ví dụ và pipeline

Ở state có hai action, probabilities $(0.5, 0.5)$ theo softmax logits; sample chọn action thứ nhất, return 4, baseline 3. Advantage sample 1; log-gradient theo hai logits là $(0.5, -0.5)$. Với $\gamma^t = 1, \alpha = 0.1$, logits thay đổi $(+0.05, -0.05)$: action tốt hơn baseline được tăng probability.

1. **Dữ liệu:** Actor hiện tại thu rollout; ghi log probabilities và terminal boundaries.
2. **Học:** Tính return/advantage; fit critic; ascent actor bằng score times advantage, giữ estimate advantage cố định trong actor gradient.
3. **Dùng và đánh giá:** Actor sinh action hoặc distribution; ghi rõ stochastic/deterministic evaluation và cùng task budget.

5. So sánh, giới hạn và ứng dụng

REINFORCE dùng MC, thường nhiễu nhưng không cần bootstrap. Actor–critic dùng critic, thêm approximation error. Baseline không cần neural value và không luôn giảm variance. Ví dụ ứng dụng: continuous-control policy sinh action distribution.

6. Tự luyện

1. Return 2, baseline 3: dấu policy update đối với sampled action thế nào?
2. Hai TD errors $1, 2$ với $\gamma\lambda = 0.8$: tính GAE tại bước đầu.
3. Vì sao không tùy ý trừ baseline phụ thuộc action trong REINFORCE?

Đáp án: phụ lục tự luyện.

7. Điều cần nhớ và đọc thêm

Policy gradient là expected gradient; estimator có noise. Baseline và bootstrap là hai vai trò khác nhau. Đọc Silver 7, Sutton–Barto §§13.2–13.5, Berkeley 5–6.

9 Policy updates hiện đại: TRPO, PPO và maximum-entropy RL

Mở rộng: Berkeley 9–10, 12–13; liên hệ Silver 7 và Boyd Ch. 5.

> **Mục tiêu học**
> Hiểu tại sao policy update lớn làm dữ liệu cũ kém phù hợp; phân biệt PPO surrogate với return objective; giải thích entropy và cơ chế SAC.

1. Hiểu nhanh

Batch của policy cũ không mô tả hoàn toàn state distribution của policy mới. Bước đổi policy lớn có thể làm hành vi xấu đi ngoài những states đã kiểm tra. TRPO/PPO kiểm soát update; maximum-entropy RL thêm entropy vào objective. Một bên điều chỉnh cách học, bên kia đổi mục tiêu.[3, Lectures 9–10, 13]

2. Bài toán được giải quyết

PPO học actor/critic từ rollouts của π_{old}. Nhiều epochs trên cùng batch không biến nó thành phương pháp replay off-policy bất kỳ. SAC dùng replay, soft critics và stochastic actor; evaluation có thể sample hoặc chọn deterministic action theo protocol.

3. Ý tưởng và công thức cốt lõi

Importance ratio trong surrogate.

$$\rho_t(\theta) = \frac{\pi_\theta(A_t \mid S_t)}{\pi_{\text{old}}(A_t \mid S_t)}. \tag{29}$$

So **xác suất của cùng action ở cùng state** dưới policy mới và policy đã thu rollout. Ratio 1 nghĩa chưa đổi probability của action đó; 1.4 nghĩa mới bằng 1.4 lần cũ. Mẫu số giữ cố định trong batch. Action được sample từ policy cũ, không sample lại mỗi lần tính ratio. Ratio reweight action samples; nó không tự sửa toàn bộ thay đổi state distribution khi policy mới được chạy.

$$L^{\text{clip}}(\theta) = \hat{\mathbb{E}}_t[\min\{\rho_t(\theta)\hat{A}_t, \text{clip}(\rho_t(\theta), 1-\epsilon, 1+\epsilon)\hat{A}_t\}]. \tag{30}$$

Với mỗi sample, tính hai mức đóng góp: $\rho_t\hat{A}_t$ và phiên bản dùng ratio bị clip vào $[1-\epsilon, 1+\epsilon]$. Chọn giá trị nhỏ hơn, rồi average batch bằng $\hat{\mathbb{E}}_t$; actor maximize kết quả. **Vì sao cần xét dấu advantage?** Với $\hat{A}_t > 0$, tăng ratio quá lớn mất lợi ích thêm; với $\hat{A}_t < 0$, giảm ratio quá nhỏ mất lợi ích thêm. ϵ là clip width ở đây. Clip làm surrogate thận trọng hơn, không buộc mọi ratios nằm trong khoảng đó.

L^{clip} là surrogate để cập nhật actor; critic có loss riêng. Cách average batch này không mặc định bằng exact discounted gradient (25); cần biết cách weighting theo thời gian khi triển khai. PPO không bảo đảm KL nhỏ ở mọi state hay return tăng mỗi epoch. TRPO dùng KL trust region và phép xấp xỉ cục bộ/natural gradient; hãy phân biệt kết quả lý thuyết với thuật toán tính gần đúng.[3, Lecture 10]

Entropy.

$$\mathcal{H}(\pi(\cdot \mid s)) = -\mathbb{E}_{A\sim\pi(\cdot|s)}[\log \pi(A \mid s)]. \tag{31}$$

Entropy đo mức phân tán của action distribution tại *một state*. Với actions rời rạc, chọn gần như chắc một action có entropy gần 0; chia đều hai actions có entropy $\log 2$. Dấu trừ đổi average log-probability thành entropy. **Có phải entropy cao là khám phá tốt?** Chưa: distribution phân tán có thể vẫn không tạo được chuỗi actions tới reward xa. Với continuous actions, đây là differential entropy; không suy nguyên các tính chất rời rạc sang nó.

$$J_{\text{ent}}(\theta) = \mathbb{E}_{\pi_\theta}\left[\sum_{t=0}^{T-1} \gamma^t\{R_{t+1} + \alpha_{\text{ent}}\mathcal{H}(\pi_\theta(\cdot \mid S_t))\}\right]. \tag{32}$$

Mỗi bước có hai thành phần được đánh giá: task reward R_{t+1} và entropy tại state đó. α_{ent} đặt mức quan trọng của entropy tương đối với reward; sau đó discount và average trajectories như return thường. Đây là **thay đổi mục tiêu tối ưu**, không chỉ thêm một mẹo optimizer. Temperature khác learning rate α: một bên đổi trade-off của objective, một bên đổi độ lớn update. Đặt temperature bằng zero chỉ bỏ entropy khỏi objective.

Bỏ entropy khỏi objective không làm actor–critic, replay và target updates của SAC tự biến thành DQN.

SAC ở mức cơ chế. Soft critic đánh giá reward cùng entropy tương lai, bootstrap theo actor action. Actor tăng soft Q và giữ entropy. Practical SAC thường dùng minimum của hai critics để hạn chế overestimation, cùng target networks/replay. Temperature có thể cố định hoặc học theo target entropy; khác learning rate.[3, Lecture 13 và discussion về SAC]

4. Ví dụ và pipeline

PPO có $\hat{A} = 2, \rho = 1.4, \epsilon = 0.2$: unclipped term 2.8, clipped term 2.4, surrogate chọn 2.4. Nếu $\hat{A} = -2$ với cùng ratio, min chọn -2.8; không được lấy clipped term một cách máy móc ở mọi dấu advantage.

1. **Dữ liệu:** PPO thu batch với policy cũ; SAC lưu replay.
2. **Học:** PPO giữ old log-probs/advantages để fit critic và tối ưu actor; SAC cập nhật soft critic, actor và temperature nếu học.
3. **Dùng và đánh giá:** Đo return/success theo cùng protocol; entropy/KL/critic chỉ hỗ trợ chẩn đoán.

5. So sánh, giới hạn và ứng dụng

PPO dùng surrogate; SAC dùng reward–entropy và replay. Hiệu quả về data có thể khác hiệu quả về thời gian; cả hai vẫn chịu lỗi reward, critic và quan sát.

6. Tự luyện

1. $\rho = 0.7, \epsilon = 0.2, \hat{A} = -2$: hai terms và PPO minimum là gì?
2. PPO clipping có bảo đảm KL nhỏ tại mọi state không?
3. Đổi entropy temperature có thể đổi policy tối ưu không?

Đáp án: phụ lục tự luyện.

7. Điều cần nhớ và đọc thêm

Surrogate, constraint và task objective là ba thứ khác nhau. Entropy temperature không phải step size. Đọc Berkeley 9–10, 12–13; gradient/duality ở Mục 19.

III. Khám phá, planning và world models

10 Exploration, bandits và thu thập thông tin

Nền tảng: Silver 9; Sutton–Barto Ch. 2. Mở rộng: Berkeley 19, 23.

> **Mục tiêu học**
> Phân biệt exploration với stochastic action; hiểu epsilon-greedy, UCB, Thompson sampling và giới hạn của intrinsic reward.

1. Hiểu nhanh

Exploitation dùng hành động hiện được đánh giá cao; exploration chấp nhận hành động khác để thu thông tin có thể hữu ích về sau. Bandit cô lập bài toán này khi action không đưa agent vào chuỗi state mới. RL đầy đủ thêm khó khăn: muốn biết một state có ích có thể cần một chuỗi action phối hợp.[1, Lecture 9]

2. Bài toán được giải quyết

Bandit input là action và scalar feedback; estimate giá trị trung bình từng arm; output rule chọn arm theo lịch sử. Trong RL, exploration còn thay visited states và data support. Metric có thể là cumulative reward/regret khi benchmark biết optimum, hoặc final policy performance/interaction cost; chúng không đồng nghĩa.

3. Ý tưởng và công thức cốt lõi

Epsilon-greedy. Chọn greedy với xác suất $1-\epsilon$; phần còn lại chọn random theo distribution đã quy định. Dễ dùng nhưng không phân biệt action ít dữ liệu với action đã biết kém. Optimistic initialization kích thích thử action ở đầu nhưng phụ thuộc initialization và nonstationarity.

$$Q_{n+1}(a) = Q_n(a) + \alpha_n[R - Q_n(a)]. \tag{33}$$

$Q_n(a)$ dự đoán reward trung bình của arm a trước update; R là reward vừa thấy. Dùng error $R - Q_n(a)$ để sửa estimate, với α_n kiểm soát mức sửa. n đếm các lần cập nhật *arm này*, không tổng lần chọn tất cả arms. Chọn step size bằng nghịch đảo số samples được tính cho cách lấy sample mean. Trong bandit không có discounted next-state value vì không cần đánh giá một chuỗi state transitions.

$$A_t \in \arg\max_a \left[Q_t(a) + c\sqrt{\frac{\ln t}{N_t(a)}} \right]. \tag{34}$$

Score của arm gồm **estimate reward hiện tại + bonus do còn ít dữ liệu**. $N_t(a)$ là số lần đã chọn arm trước t; count nhỏ cho bonus lớn, nên action chưa chắc tốt vẫn đáng thử. $\ln t$ làm một arm lâu chưa được thử có cơ hội được xét lại; $c > 0$ quyết định cường độ bonus. Arm count zero cần ưu tiên thử hoặc xử lý riêng, không chia trực tiếp cho zero. Bonus là rule chọn arm, không phải reward thật của task.

UCB dùng bonus cho uncertainty từ số mẫu trong setting bandit. Thompson sampling duy trì posterior về arm values, sample một giả thuyết, chọn action tối ưu theo giả thuyết đó. Random action không đồng nghĩa posterior sampling.

Gradient bandit. Học preferences cho actions bằng softmax và reward tương đối baseline; preference ở đây là tham số chọn arm, không phải human preference hoặc MORL utility.[2, §2.8]

Exploration trong deep RL. Berkeley phân biệt bonus từ count hoặc density, curiosity từ novelty hoặc prediction error và kỹ năng không giám sát. Prediction error cao có thể do randomness không thể học, nên bonus dễ bị kéo tới "noisy TV". Deep exploration cần hành vi nhất quán nhiều bước; action noise từng bước có thể không tới được reward xa.[3, Lectures 19, 23]

4. Ví dụ và pipeline

Hai arms có estimates $1.0, 0.8$, counts $100, 1$, $t = 101, c = 1$. UCB scores xấp xỉ $1.215, 2.948$, nên thử arm thứ hai dù mean estimate thấp hơn. Đây là một phép tính theo rule UCB, không khẳng định arm thứ hai thật sự tốt hơn.

1. **Dữ liệu:** Behavior được chọn để vừa tìm reward vừa lấy thông tin.
2. **Học:** Cập nhật estimates/posterior hoặc exploration representation; intrinsic signals là thành phần thuật toán được nêu rõ.
3. **Dùng và đánh giá:** Đánh giá task reward gốc, discovery/sample efficiency; không báo bonus reward như task success.

5. So sánh, giới hạn và ứng dụng

High entropy, random actions và information-seeking là các cơ chế khác nhau. Bandit uncertainty không tự mô tả dynamics uncertainty. Coverage trong asymptotic theorem không xác định đủ interaction budget thực tế. Ví dụ ứng dụng minh họa: chọn chiến lược hoặc cấu hình thí nghiệm có feedback, rồi mở rộng sang decision sequence.

6. Tự luyện

1. Vì sao epsilon-greedy thử lại cả action đã biết chắc kém?
2. Nếu prediction error cao vì sensor noise không thể dự đoán, curiosity có thể bị gì?
3. Bandit "không có chuyển state" khác gì navigation tới reward bị khóa sau nhiều hành động?

Đáp án: phụ lục tự luyện.

7. Điều cần nhớ và đọc thêm

Exploration là cơ chế tạo data hữu ích, không chỉ entropy. Đọc Silver 9, Sutton–Barto §§2.3–2.9 và Berkeley 19/23.

11 Learning, planning, Dyna và model-based RL

Nền tảng: Silver 8; Sutton–Barto Ch. 8. Mở rộng: Berkeley 15–16.

> **Mục tiêu học**
> Phân biệt real/simulated experience; hiểu Dyna, search và MPC; nhận ra model exploitation.

1. Hiểu nhanh

Model giúp suy nghĩ về hành động chưa thực hiện trong environment thật. Planning có thể cập nhật value bằng simulated transitions hoặc tìm action trực tiếp bằng rollouts. Learned model không hoàn hảo; optimizer có thể tìm những action khai thác lỗi model. Model-based RL vì vậy có cả bài toán học model và bài toán ra quyết định dựa trên model đó.[1, Lecture 8]

2. Bài toán được giải quyết

Input: real transitions và reward/task; output: model cùng policy/value hoặc controller có planner. Khi deployment dùng MPC, current state đi vào learned model/planner để tạo action; vì vậy inference cần model và search budget. Nếu model chỉ tạo training data cho actor, deployment có thể chỉ dùng actor.

3. Ý tưởng và công thức cốt lõi

Dyna. Từ một real transition, update Q như model-free learning và cập nhật model. Sau đó sample state-action đã có dữ liệu, lấy simulated next state/reward từ model, rồi tiếp tục Q updates. "Model-free update rule" không biến toàn bộ agent thành model-free nếu nó dùng model để tạo target data.[2, §§8.1–8.3]

Học dynamics. Với density model $p_\eta(s' \mid s, a)$:

$$L(\eta) = -\mathbb{E}_{\mathcal{D}}[\log p_\eta(S_{t+1} \mid S_t, A_t)]. \tag{35}$$

Giữ (S_t, A_t) và hỏi model gán likelihood bao nhiêu cho next state *đã xảy ra* S_{t+1}. Nếu likelihood thấp, negative log-likelihood lớn và optimizer phải sửa model parameters η. Average theo $\mathcal{D}$ để học từ nhiều transitions. Với next states liên tục, p_η là density. Công thức học khả năng dự đoán chuyển tiếp; nó không tối ưu trực tiếp return của controller và cũng không bảo đảm model đúng ngoài data đã thấy.

Deterministic model thường fit next-state error; distributional model diễn tả aleatoric randomness. Ensemble disagreement có thể là proxy epistemic uncertainty, không mặc định posterior được hiệu chuẩn.

MPC. Tại mỗi bước, tối ưu một action sequence dài H trong model, execute action đầu, quan sát state mới rồi replan:

$$(a_0^*, \dots, a_{H-1}^*) \in \arg\max_{a_{0:H-1}} \mathbb{E}_{\hat{p}_\eta}\left[\sum_{k=0}^{H-1} \gamma^k R_{t+k+1} \mid S_t = s\right]. \tag{36}$$

Ta đang ở s: thử nhiều sequences $a_0, \dots, a_{H-1}$, dùng learned model $\hat{p}_\eta$ dự đoán rewards và chọn sequence có expected discounted sum lớn nhất. a_0 là action sẽ làm ngay, không phải action lúc bắt đầu episode; H là số bước nhìn trước. Chỉ execute action đầu, quan sát state thật mới rồi tối ưu lại. Vì **dự đoán có thể sai**, replanning giúp phản hồi theo

quan sát mới; search vẫn chỉ cho approximate solution nếu solver không tìm được global optimum.

Replanning biến việc execute thành feedback control dù mỗi search tối ưu open-loop action sequence. Closed-loop planning tối ưu policy phụ thuộc state tương lai là đối tượng khác. CEM/random shooting chỉ là cách tìm approximate optimizer; không chứng minh tìm global optimum.

Search và planning backups. Prioritized sweeping ưu tiên backup có khả năng truyền thay đổi lớn; sample/expected backups khác nhau ở chi phí tính và noise. Tree search/MCTS tập trung compute vào lựa chọn tại state hiện tại. Không suy ra search có learned model thì model phải là neural network.[2, §§8.4–8.11]

4. Ví dụ và pipeline

Controller có thể dùng 100 rollouts trong model để chọn một action rồi thực hiện đúng một transition thật. Interaction cost là 1, model rollout cost là $100H$ bước simulated; không được gộp thành cùng một con số "samples". Nếu model dự đoán một lối đi qua tường, planner có thể ưu tiên chính lối đó vì estimated return cao.

1. **Dữ liệu:** Thu dữ liệu thật; fit model và reward predictor nếu cần.
2. **Học:** Plan hoặc tạo short synthetic rollouts để học value/actor; thu thêm dữ liệu nơi controller mới hoạt động.
3. **Dùng và đánh giá:** MPC replan sau observation mới; actor-only deployment không chạy planner. Đo interaction, model error và total compute riêng.

5. So sánh, giới hạn và ứng dụng

Model-free cần critic/policy chính xác cho quyết định; model-based thêm khả năng phản thực nhưng chịu dynamics error. Rollout dài tích lũy error; rollout ngắn từ real replay states như MBPO giảm một nguồn error nhưng không xóa model bias. Minh họa application là control nơi real interactions đắt; model training và planning vẫn tốn compute.[3, Lecture 16]

6. Tự luyện

1. Trong Dyna, simulated transition có tăng số environment interactions thật không?
2. Vì sao one-step prediction error nhỏ vẫn có thể cho policy tệ?
3. MPC tối ưu open-loop sequence nhưng execute với replanning: phần nào là feedback?

Đáp án: phụ lục tự luyện.

7. Điều cần nhớ và đọc thêm

Learning học thành phần chưa biết; planning dùng model để cải thiện quyết định. Model usefulness phải kiểm tra theo task và state distribution. Đọc Silver 8, Sutton–Barto §§8.1–8.6, 8.10–8.11, Berkeley 15–16.

12 World models, latent states và partial observability

Mở rộng: Berkeley 16, phần latent-state models; Berkeley 14, phần POMDP. Liên hệ Silver 1 và 8.

> **Mục tiêu học**
> Mô tả world model ở mức representation/dynamics/reward; phân biệt planning trong model với học actor trong imagination.

1. Hiểu nhanh

World model học biểu diễn và quy luật chuyển biến để phục vụ quyết định. Khi quan sát là ảnh hoặc không đủ Markov, agent có thể dùng lịch sử để suy ra latent state. Model không nhất thiết phải tái dựng từng pixel hoàn hảo; quan trọng là giữ được thông tin cần để đánh giá action và reward tương lai.[3, Lecture 16]

2. Bài toán được giải quyết

Input: observation/action/reward sequences, có thể partial observability; output: encoder/state estimator, latent dynamics, reward predictor và actor/critic hoặc planner. Training cần thông tin theo thời gian; shuffle các frame độc lập có thể làm mất temporal structure. Deployment cần history representation nếu một observation không đủ.

3. Ý tưởng và công thức cốt lõi

Ba vai trò model. Latent dynamics dự đoán next latent state, observation model liên hệ latent với quan sát, reward model dự đoán task feedback. Encoder/filtering model suy luận latent từ dữ liệu. Với POMDP, latent estimate nên dùng thông tin quá khứ; single-frame encoder không được bảo đảm tạo Markov state.

Conditional factor	Vai trò
$p_\eta(z_{t+1} \mid z_t, a_t)$	Dynamics: dự đoán latent kế tiếp.
$p_\eta(o_t \mid z_t)$	Observation: mô hình hóa quan sát từ latent.
$p_\eta(r_{t+1} \mid z_t, a_t)$	Reward: dự đoán feedback của action hiện tại.

Đọc ba phân phối như ba câu hỏi. Biết latent state/action thì next latent sẽ là gì? Biết latent thì observation có thể trông thế nào? Biết latent/action thì reward bước tới có thể là bao nhiêu? η là tham số model, z_t là latent, o_t là observation. Các hàng mô tả distributions chứ không phải ba losses. Learned latent cần giữ thông tin cho control; ba factors cơ bản này chưa quy định toàn bộ kiến trúc world model.

Một world model cụ thể có thể bổ sung recurrent deterministic state, stochastic state hoặc conditioning khác. Hãy xem nó giữ ký ức ở đâu và latent được suy luận từ thông tin nào; tên "latent state tự nó không bảo đảm representation đủ Markov.

Hai cách dùng model. (i) Planning: search action sequences trong latent dynamics, rồi execute/replan như MPC. (ii) Imagination learning: tạo synthetic trajectories để update actor/critic; khi dùng, actor sinh action từ inferred latent state. Model có thể phục vụ representation learning mà không được rollout tại deployment.

Representation phục vụ điều gì? Reconstruction tốt kiểm tra giữ thông tin observation; reward/value prediction kiểm tra task relevance. Hai tiêu chí có thể khác nhau. Nếu representation bỏ vật cản nhỏ nhưng quan trọng, pixel error tổng thấp vẫn không bảo đảm policy tốt. Nếu dynamics chỉ fit visited states, imagined actions ngoài support

có thể gây sai.

4. Ví dụ và pipeline

Hai ảnh chụp một vật ở cùng vị trí nhưng vật có vận tốc khác nhau. Single-frame representation khó phân biệt next state; history hai hoặc nhiều frame có thể cung cấp motion information. Đây là ví dụ observability, không yêu cầu mọi task phải dùng video input.

1. **Dữ liệu:** Thu sequences và boundary; tách dữ liệu kiểm tra theo trajectory/điều kiện phù hợp.
2. **Học:** Học encoder/dynamics/reward theo objective của model cụ thể; dùng model cho planning hoặc synthetic actor–critic updates.
3. **Dùng và đánh giá:** Suy luận latent từ observation/history mới; dùng actor hoặc planner. Đánh giá behavior trong environment thật, không chỉ reconstruction loss.

5. So sánh, giới hạn và ứng dụng

Model-based RL với state vectors không nhất thiết cần latent world model. Autoencoder có latent nhưng không tự động là world model cho control. Filtering dùng thông tin hiện tại/quá khứ; smoothing có thể dùng future observations, nên không được đưa future-only information vào deployment. Không giả định latent Markov vì nó được gọi là “state”.

6. Tự luyện

1. Reconstruction loss thấp có đủ chứng minh reward prediction đúng không?
2. Khi actor được train trong imagination, deployment có nhất thiết phải search action sequences không?
3. Vì sao encoder dùng future observations khi train cần được xử lý cẩn thận lúc deployment?

Đáp án: phụ lục tự luyện.

7. Điều cần nhớ và đọc thêm

World model là thành phần học; planning/imagination là cách sử dụng nó. Model quality phải gắn với decision quality. Đọc Berkeley 16 slides 29–38; latent learning ở Mục 15.

IV. Học từ dữ liệu và suy luận

13 Imitation learning và inverse RL

Mở rộng: Berkeley 2–3; Berkeley 13, phần IRL và adversarial imitation.

> **Mục tiêu học**
> Phân biệt học action từ demonstrations với học reward từ behavior; hiểu covariate shift và reward ambiguity.

1. Hiểu nhanh

Imitation bắt đầu bằng hành vi mẫu, không nhất thiết có reward rõ ràng. Behavioral cloning (BC) fit action giống expert trên những state có trong dữ liệu. Inverse RL hỏi reward nào có thể giải thích behavior; sau đó dùng reward đó cho control. Học giống hành động quan sát được và suy ra mục tiêu của người hành động là hai câu hỏi khác nhau.[3, Lectures 2–3, 13]

2. Bài toán được giải quyết

BC input là pairs (s, a) expert, output policy. Khi deployment, policy đến states do chính hành động của nó tạo ra. IRL input thường là demonstrations cùng assumptions về dynamics/expert optimality, output reward và có thể policy. Không mặc định demonstration là tối ưu tuyệt đối hoặc mọi reward đều identifiable.

3. Ý tưởng và công thức cốt lõi

$$L_{\mathrm{BC}}(\theta) = -\mathbb{E}_{(s,a)\sim\mathcal{D}}[\log \pi_\theta(a \mid s)]. \tag{37}$$

Mỗi pair (s, a) trong demonstrations nói expert đã chọn gì tại state đó. BC làm policy gán likelihood cao hơn cho action mẫu bằng cách minimize negative log-likelihood. Dấu trừ biến bài toán maximize likelihood thành minimize loss. Công thức không chứa reward hay next-state value: learner được dạy *action*, không trực tiếp được dạy hậu quả dài hạn. Với continuous actions dùng policy density phù hợp; regression deterministic là một lựa chọn khác. Khi chạy thật, learner có thể tới states ít có trong demonstrations.

Loss này fit expert actions trên expert-state distribution. Một lỗi nhỏ làm policy rời distribution đó; ở state lạ, lỗi mới có thể lớn hơn và tích lũy qua thời gian. Validation likelihood trên expert data không đo trực tiếp closed-loop success.

Dataset aggregation. DAgger đưa learner tới states do nó thăm, xin action label từ expert, rồi aggregate và retrain. Nó cần quyền gọi expert tại những state đó; khác với chỉ có logged demonstrations cố định.[3, Lecture 3]

Inverse RL. Một behavior có thể phù hợp nhiều reward functions; reward constant hoặc transformations phù hợp có thể không xác định một mục tiêu duy nhất. Maximum-entropy IRL mô hình hóa stochastic demonstrations và regularization/structure; theorem về reward recovery phải đọc cùng assumptions. Không gọi reward được fit là "true human reward" nếu chưa có identification argument.

Adversarial imitation. Discriminator phân biệt expert với learner behavior; policy được tối ưu theo tín hiệu discriminator. Nhìn ở mức occupancy giúp hiểu cơ chế matching. Việc có adversarial objective không tự chứng minh reward học được transfer sang dynamics mới.[3, Lecture 13]

4. Ví dụ và pipeline

Expert lái ở giữa làn, dataset hiếm khi có state sát mép. BC lệch nhẹ rồi gặp state sát mép; dataset không cho biết cách hồi phục. DAgger có thể xin expert action hồi phục ở state đó. Với IRL, việc expert đi đường ngắn có thể do tiết kiệm thời gian, năng lượng hoặc nhiều reward kết hợp; trajectory đơn lẻ không phân biệt chúng.

1. **Dữ liệu:** Thu demonstrations, ghi information agent thực sự nhìn thấy và expert access.
2. **Học:** BC fit action likelihood; DAgger bổ sung labels từ learner states; IRL fit reward với control/inference loop.
3. **Dùng và đánh giá:** Execute learned policy và đo success/return; reward recovery cần kiểm tra transfer hoặc assumptions riêng.

5. So sánh, giới hạn và ứng dụng

BC là supervised objective trên sequential task; IRL là reward inference; preference learning dùng comparisons thay vì nhất thiết expert trajectories (Mục 17). Imitation có thể khởi tạo RL, nhưng không vì thế mọi BC model là RL-trained policy. Robotics là ví dụ application cần demonstrations và recovery behavior.

6. Tự luyện

1. BC validation loss giảm nhưng robot success giảm: có mâu thuẫn không?
2. DAgger có dùng được nguyên bản nếu không được hỏi expert ở state mới không?
3. Expert behavior có xác định reward duy nhất không?

Đáp án: phụ lục tự luyện.

7. Điều cần nhớ và đọc thêm

Demonstrations cung cấp actions, không tự cung cấp mục tiêu duy nhất. Đọc Berkeley 2–3 và phần IRL/adversarial ở Lecture 13.

14 Offline RL, data support và offline-to-online

Mở rộng: Berkeley 17–18. Liên hệ Sutton–Barto Ch. 11.

> **Mục tiêu học**
> Phân biệt offline với off-policy; giải thích OOD action values, CQL/IQL và giới hạn của offline evaluation.

1. Hiểu nhanh

Offline RL học policy từ dataset cố định mà không được thu thêm interaction trong giai đoạn học. Dataset có reward nên không chỉ sao chép behavior như BC. Nhưng policy optimizer có thể chọn action không được dữ liệu hỗ trợ; critic lại không biết chắc value của action đó. Khó khăn nằm ở support và counterfactual estimation, không chỉ vì batch nhỏ.[3, Lecture 17]

2. Bài toán được giải quyết

Input: $\mathcal{D}$ gồm logged transitions/episodes, có thể từ nhiều policies; output policy. Objective vẫn là return trong environment, không phải maximize average reward trong dataset. Nếu interaction được phép sau offline phase, setting trở thành offline-to-online; online budget cần ghi riêng.

3. Ý tưởng và công thức cốt lõi

OOD bootstrapping. Q target chọn max hoặc actor action ở next state. Nếu action ấy ít/không xuất hiện trong data, network có thể đánh giá quá cao. Error được bootstrapped lùi và actor tiếp tục chọn action giả tốt. Dataset large không bảo đảm coverage cho mọi action policy mới dùng.[3, Lectures 17–18]

Họ phương pháp	Cơ chế cần hiểu, không gán chung một loss
Policy constraints / regularization	Hạn chế policy rời behavior/data support; BC-like terms hoặc divergence có trade-off giữa giữ data và cải thiện reward.
CQL	Điều chỉnh critic theo hướng conservative: giảm value trên actions rộng hơn data, đối trọng bằng values tại data actions. Lower-bound claims phụ thuộc formulation và assumptions.
IQL	Fit value bằng expectile từ data action Q-values; fit Q với next-state value; extract actor qua advantage-weighted regression. Tránh dùng OOD actions trực tiếp trong Q backup.
Model-based offline	Học model từ logged data, kiểm soát uncertainty hoặc conservatism khi rollout. Model không tạo được knowledge chắc chắn ngoài support.

Các họ offline methods đang can thiệp ở **những chỗ khác nhau**. Policy constraint hạn chế behavior mới; CQL sửa cách critic chấm actions; IQL tạo value/Q targets từ data actions rồi extract policy; model-based methods thêm predicted transitions nhưng phải kiểm soát model error. Vì vậy không thay tất cả bằng một loss chung. Điểm cần hỏi khi đọc method là: nó đang dùng data thật để xác nhận điều gì, và phần nào vẫn là ngoại suy?

Offline policy evaluation (OPE). Muốn ước lượng policy mới từ logs mà không execute nó. Với cùng initial distribution/dynamics, có behavior probabilities và coverage,

trajectory importance sampling dùng:

$$\mathbb{E}_\pi[G_0] = \mathbb{E}_\mu\left[\left(\prod_{t=0}^{T-1}\rho_t\right)G_0\right]. \tag{38}$$

Dữ liệu được sinh bởi μ, nhưng ta muốn expected return dưới π. Product ratios sửa trọng lượng *cả trajectory*: một trajectory có các action được target ưa hơn sẽ được nhấn mạnh tương ứng. Nhân weight đó với observed G_0, rồi average dưới behavior distribution. Đẳng thức cần cùng initial distribution/dynamics, target trajectory support và kỳ vọng hữu hạn phù hợp. **Đẳng thức đúng có đồng nghĩa logs đủ tốt không?** Không: products có thể rất nhiễu, hoặc data không bao phủ trajectories target cần.

Identity không nói estimator hữu hạn mẫu đáng tin: product weights có thể cực kỳ nhiễu. Fitted Q evaluation dùng bootstrapping để đánh giá policy cố định và chịu assumptions về support/approximation. Không có một metric từ logs duy nhất bảo đảm performance trong mọi environment chưa biết.

Offline-to-online. Offline initialization có thể giảm interactions mới. Conservative values hoặc policy constraints mạnh có thể làm giai đoạn đầu online mất thời gian thích nghi. Berkeley 18 phân tích replay trộn offline/online và actor priors; phải report reward/interaction của toàn bộ phase thay vì chỉ đoạn thuận lợi.

4. Ví dụ và pipeline

Dataset ở s chỉ có action L nhận reward 1 rồi terminal; không có R. Có hai environment hoàn toàn khớp dataset: ở một environment R nhận 10, ở environment khác R nhận -10. Logged data không phân biệt hai trường hợp. Đây là giới hạn thông tin, không sửa được chỉ bằng optimizer tốt hơn.

1. **Dữ liệu:** Kiểm tra dataset origin, trajectories, rewards, task và support; train/validation tránh rò rỉ trajectory.
2. **Học:** Fit critic/policy theo offline mechanism cụ thể; không tự thêm interaction rồi vẫn gọi thuần offline.
3. **Dùng và đánh giá:** OPE nếu chỉ có logs; environment evaluation nếu protocol cho phép. Đóng băng model/checkpoint khi báo metric.

5. So sánh, giới hạn và ứng dụng

Off-policy online algorithm vẫn có thể thu data mới để sửa sai; offline algorithm không có quyền đó. BC không dùng reward để cải thiện action ngoài demonstrations. Offline task phù hợp với logs điều khiển/robotics, nhưng dataset safety không bảo đảm learned policy safety ở states mới.

6. Tự luyện

1. Dataset có một triệu transitions nhưng chỉ một action ở mỗi state: có đủ để đánh giá mọi policy không?
2. Q-learning online với replay có phải offline RL không?
3. Hai importance ratios bằng 2 và 0.5: trajectory weight là gì; một trajectory này có chứng minh estimator ổn định không?

Đáp án: phụ lục tự luyện.

7. Điều cần nhớ và đọc thêm

Offline là giới hạn quyền lấy data mới; off-policy là quan hệ giữa policies. Coverage là vấn đề thông tin. Đọc Berkeley 17–18, liên hệ importance sampling ở Sutton–Barto §5.5.

15 Variational inference và control as inference

Mở rộng: Berkeley 11–13. Nền xác suất và tối ưu hóa: Boyd Ch. 3, 5.

> **Mục tiêu học**
> Hiểu posterior, ELBO và KL; phân biệt likelihood objective của model với return objective của controller.

1. Hiểu nhanh

Latent models có biến ẩn, nên likelihood của observation phải marginalize latent states. Variational inference dùng distribution dễ xử lý để xấp xỉ posterior. Control as inference xây một probabilistic model có thêm thông tin về optimality, rồi liên hệ action inference với reward và entropy. Đây là cách derive phương pháp, không khẳng định environment thật sinh hành vi "tối ưu" theo cùng probabilistic model.[3, Lectures 11–13]

2. Bài toán được giải quyết

Latent learning input là observations và model family; output model parameters η, approximate posterior parameters ϕ. Control input còn có reward/dynamics; output action distribution/policy. ϕ trong mục này chỉ inference parameters; critic vẫn dùng w như toàn tài liệu.

3. Ý tưởng và công thức cốt lõi

Với observation o, latent z, generative model $p_\eta(o, z)$ và inference distribution $q_\phi(z \mid o)$:

$$\mathcal{L}(\eta, \phi; o) = \mathbb{E}_{q_\phi(z|o)}[\log p_\eta(o, z) - \log q_\phi(z \mid o)]. \tag{39}$$

Biến z bị ẩn, nên ta dùng $q_\phi(z \mid o)$ để đề xuất những giải thích latent cho observation o. Với mỗi đề xuất, so log joint $p_\eta(o, z)$ với log của distribution đề xuất, rồi average theo q_ϕ. Kết quả là ELBO $\mathcal{L}$, một đại lượng ta có thể maximize để học model/inference. Trừ $\log q_\phi$ liên hệ với entropy của approximate posterior; vì vậy công thức không chỉ thưởng một latent có joint probability cao. η thuộc generative model, ϕ thuộc inference model.

$$\log p_\eta(o) = \mathcal{L}(\eta, \phi; o) + D_{\mathrm{KL}}(q_\phi(z \mid o) \,\|\, p_\eta(z \mid o)). \tag{40}$$

Đẳng thức trả lời: **ELBO còn cách log-likelihood thật bao xa?** Khoảng chênh đúng bằng KL giữa approximate posterior và true posterior. KL không âm nên $\mathcal{L} \le \log p_\eta(o)$; nếu hai posteriors trùng gần như khắp nơi, bound bằng log-likelihood. Với model η cố định, giảm KL làm bound chặt hơn. Khi cùng học cả η và ϕ, cả likelihood và posterior đều có thể đổi; không diễn giải mọi tăng ELBO chỉ là giảm inference error.

Với sequence world model, latent variables liên kết theo thời gian; ELBO phải dùng đúng factorization/conditioning của model ấy. Công thức một observation ở trên giúp hiểu nguyên lý, chưa quy định đầy đủ loss của mọi world model.

KL. KL bằng zero khi hai distributions trùng gần như khắp nơi, không đối xứng và không phải distance metric đúng nghĩa. Forward/reverse KL có thể ưu tiên fit khác nhau. KL giữa policy và reference có thể làm regularizer; KL giữa approximate và true posterior nằm trong inference identity, không cùng đối tượng.

Control as inference. Reward được đưa vào optimality likelihood theo model được định nghĩa. Trong setting thích hợp, inference/variational derivation đưa tới soft value và

maximum-entropy behavior ở Mục 9. Với stochastic dynamics, naive exact conditioning có thể tạo "optimistic transitions" như thể controller thay được dynamics; variational formulation phải giữ dynamics thật cố định.[3, Lecture 13, slides 7–12]

4. Ví dụ và pipeline

Nếu $\log p_\eta(o) = -5$ và posterior KL 2, ELBO là -7. Nếu inference model fit tốt hơn làm KL còn 0.5 với generative model cố định, ELBO thành -5.5. Điều này chứng minh inference bound chặt hơn, không tự chứng minh policy return tốt hơn.

1. **Dữ liệu:** Chọn generative factors và variables observed/latent.
2. **Học:** Optimize ELBO/model inference theo assumptions; controller dùng soft policy updates nếu derivation cho setting ấy.
3. **Dùng và đánh giá:** Suy luận posterior/latent từ information hiện có; evaluate likelihood và behavior bằng metrics riêng.

5. So sánh, giới hạn và ứng dụng

Exact inference có thể intractable; variational approximation đưa thêm approximation bias. Evidence likelihood không phải evidence về task success. Đọc probabilistic derivation cần kiểm tra dynamics được giữ cố định, reward scaling, entropy temperature và observations available at decision time.

6. Tự luyện

1. Log likelihood -3, posterior KL 1.2: ELBO là bao nhiêu?
2. Vì sao KL không phải distance đối xứng?
3. Một derivation cho phép đổi stochastic transition probabilities để có reward cao: có đúng control problem ban đầu không?

Đáp án: phụ lục tự luyện.

7. Điều cần nhớ và đọc thêm

ELBO học probabilistic model; return chọn behavior; control-as-inference nối chúng dưới một setting được chỉ rõ. Đọc Berkeley 11–13, không xem inference là một objective chung cho mọi RL.

V. Khái quát hóa và nghiên cứu hiện đại

16 Multi-task, goal-conditioned và hierarchical RL

Mở rộng: Berkeley 24. Liên hệ Silver 2, 8 và Sutton–Barto Ch. 3.

> **Mục tiêu học**
> Phân biệt task context, goal và preference; hiểu goal relabeling, successor features và options ở mức cơ chế.

1. Hiểu nhanh

Multi-task RL chia sẻ kiến thức giữa tasks; goal-conditioned policy nhận goal cần đạt; hierarchy chọn hành vi kéo dài nhiều bước. Chia sẻ có thể giúp transfer hoặc làm tasks cản trở nhau. Những mở rộng này không tự tạo một reward vector.[3, Lecture 24]

2. Bài toán được giải quyết

State/observation, context hoặc goal và trajectories nhiều tasks dùng để học conditional policy. Phân biệt train/test task distributions, zero-shot transfer và adaptation có data mới. g chỉ điều kiện muốn đạt, không mặc định là MORL preference weights.

3. Ý tưởng và công thức cốt lõi

$$q_\pi(s, a, g) = \mathbb{E}_\pi[G_t \mid S_t = s, A_t = a, g]. \tag{41}$$

Cùng state/action có thể tốt cho goal này nhưng kém cho goal khác. Vì thế giữ thêm g trong điều kiện: return được tính với reward của goal đó, và policy tiếp tục nhận goal, $\pi_\theta(a \mid s, g)$. Ví dụ cùng một bước sang trái có thể đưa ta gần g_1 nhưng xa g_2. Đây là value được điều kiện hóa theo task; g không tự là một vector preference weights của MORL.

Task augmentation. Với task context cố định trong episode, có thể xem (s, g) hoặc $(s, \text{context})$ là joint state của MDP khi nó chứa đủ thông tin Markov. Nếu context bị ẩn hoặc đổi động, cần chỉ rõ observation/dynamics của context.

Goal relabeling. Trajectory không đạt requested goal có thể đạt goal khác. Relabel goal, tính lại reward theo goal mới và đưa transition vào off-policy learner. Cần reward xác định được từ trajectory/state; không giữ reward cũ khi đổi goal nếu reward thật đã đổi. Với stochastic environments, relabeling không được bảo đảm unbiased trong mọi formulation.

Successor features. Với linear reward $\mathcal{R}_s^a = \varphi(s, a)^\top u$, u là reward weights và φ là feature vector:

$$\psi_\pi(s, a) = \mathbb{E}_\pi \left[\sum_{k=0}^{\infty} \gamma^k \varphi(S_{t+k}, A_{t+k}) \mid S_t = s, A_t = a \right]. \tag{42}$$

$\psi_\pi(s, a)$ không cộng reward scalar mà cộng **vectors đặc trưng reward** φ dự kiến gặp trong tương lai. Mỗi thành phần theo dõi một loại đặc trưng, có cùng discount γ^k. Expectation cố định state/action đầu, sau đó theo π. Với bounded features và $\gamma < 1$, tổng

vô hạn được kiểm soát. Vì đang giữ policy cố định, successor features mô tả tương lai của policy ấy, không của mọi policies cùng lúc.

$$q_\pi(s,a) = \psi_\pi(s,a)^\top u. \tag{43}$$

Nếu reward một bước tuyến tính theo φ với weights u, thì expected total reward là dot product của **expected total features** với cùng u. Về đại số, weights cố định có thể đưa ra ngoài tổng và expectation. Nhờ vậy đổi weights cho một task mới có thể tính lại value mà không học lại ψ_π của policy đó. u là reward weights, khác critic parameters w. Điều này đổi cách chấm cùng behavior, chưa tìm policy tối ưu mới.

Khi đổi reward weights, bước tính lại dot product cho value mới của cùng policy; muốn behavior tốt hơn có thể cần chọn hoặc học policy khác. Generalized policy improvement dùng nhiều candidate policies/values để hỗ trợ việc đó, dưới assumptions riêng về accuracy và policy set.

Hierarchy và options. Option gồm tập states được bắt đầu, policy nội bộ và quy tắc dừng. High-level policy chọn option, low-level chạy nhiều primitive steps. Cộng rewards bên trong rồi discount theo *duration*, không chỉ một lần cho cả option. Với subgoals, low-level policy cần thật sự đạt được chúng.

4. Ví dụ và pipeline

Goal-conditioned navigation được yêu cầu tới ô g_1 nhưng trajectory kết thúc tại g_2. Ta có thể relabel thành task g_2 nếu reward được tính lại đúng. Với successor features $\psi = (2,1)^\top$, reward weights $u = (1,-0.5)^\top$, value của policy đã cho là 1.5; đổi u đổi value mà không nhất thiết đổi dynamics features.

1. **Dữ liệu:** Sample tasks/goals, ghi context cùng transitions; chọn option/subgoal nếu có hierarchy.
2. **Học:** Học shared conditional actor/critic; relabel theo rule hợp lệ; học low/high-level modules đúng time scales.
3. **Dùng và đánh giá:** Nhận goal/context khi dùng; đo từng task và held-out tasks, interaction/adaptation cost.

5. So sánh, giới hạn và ứng dụng

Multi-task có thể có scalar reward riêng từng task; MORL có vector rewards và solution concept khác. Goal-reaching không bao phủ mọi task. Hierarchy giảm decision horizon ở cấp cao nhưng thêm khó khăn thiết kế/học skills. Robotics/navigation là application minh họa; feasibility phải xét data goal coverage và low-level reachability.

6. Tự luyện

1. $\psi = (3,2)^\top$, $u = (1,-1)^\top$: tính q_π.
2. Đổi reward weights có bảo đảm policy cũ tối ưu không?
3. Relabel goal mà giữ nguyên reward không tương ứng có lỗi gì?

Đáp án: phụ lục tự luyện.

7. Điều cần nhớ và đọc thêm

Context/goal điều kiện hóa policy; successor features tái sử dụng prediction; hierarchy thay đơn vị quyết định. Đọc Berkeley 24. MORL cần nguồn chuyên biệt ngoài phạm vi bốn tài liệu này.

17 RL với sequences và LLMs

Mở rộng: Berkeley 14, phần LLM RL và preference/verifier rewards. Liên hệ Mục 8.

> **Mục tiêu học**
> Ánh xạ token generation vào sequential decisions; phân biệt SFT, RLHF, verifier reward và group baseline.

1. Hiểu nhanh

Language model có thể được nhìn như policy sinh token theo prefix. Một completion là một episode; task score hoặc learned reward cung cấp tín hiệu để cải thiện behavior. Cơ chế policy gradient vẫn là tăng/giảm probability những actions đã sample, nhưng actions là tokens và reward thường tới cuối sequence.[3, Lecture 14]

2. Bài toán được giải quyết

State chứa prompt và prefix; action là next token; transition thêm token; terminal là end-of-sequence hoặc task-defined end. Input khi train gồm prompts, completions và reward/preference/verifier data; output là post-trained policy, đôi khi reward model/critic. Deployment thường chỉ cần policy và decoding rule, không cần reward model gọi ở mỗi token.

3. Ý tưởng và công thức cốt lõi

Sequence likelihood. Với prompt x, completion $a_{0:T-1}$:

$$\pi_\theta(a_{0:T-1} \mid x) = \prod_{t=0}^{T-1} \pi_\theta(a_t \mid x, a_{0:t-1}). \tag{44}$$

Xác suất sinh cả completion là product xác suất sinh từng token **khi đã biết prompt và các tokens trước nó**. Sau mỗi token, prefix đổi nên điều kiện của factor kế tiếp cũng đổi. Tại $t = 0$, prefix rỗng, chỉ có prompt x. Đây là chain rule của autoregressive policy, không giả định các tokens độc lập. Nhìn theo RL, state là $(x, a_{0:t-1})$, action là a_t; reward cuối completion có thể được gán credit ngược về nhiều actions.

SFT và RL. SFT fit likelihood của reference responses, tương tự BC. RL sample responses từ policy và dùng đánh giá outcomes làm tín hiệu. Terminal correctness reward khác với process reward đánh từng bước suy luận; cần xác minh tín hiệu có đúng task mong muốn không.

Preferences và verifiers. Reward learning từ pairwise preferences thường dùng Bradley–Terry likelihood theo chênh lệch reward scores. Verifier cho reward dựa trên kết quả kiểm tra được; verifier không hoàn hảo vẫn có thể bị exploit. Learned reward không tự động phản ánh đúng preference ở ngoài training distribution.

PPO/GRPO ở mức cơ chế. PPO dùng ratio và advantage như Mục 9. Critic/value baseline có thể cần thêm model; group-relative baseline dùng nhiều completions cho cùng prompt thay cho learned value baseline. Berkeley gọi cơ chế group-based version là GRPO; normalization, clipping, KL handling và token weighting cần đọc đúng variant, không có một loss phổ quát cho mọi implementation.

Reference policy. Policy cũ π_{old} dùng trong importance ratio và reference model π_{ref} dùng để regularize là hai vai trò riêng. Nếu chúng trùng ở một thời điểm không có nghĩa

chúng luôn đồng nhất. Regularization giữ behavior gần reference có thể đổi optimization trade-off, không sửa reward sai.

4. Ví dụ và pipeline

Một prompt có ba completions với correctness scores $1, 0, 0$. Baseline group mean là $1/3$, centered scores là $2/3, -1/3, -1/3$. Đây là ví dụ tín hiệu tương đối, **chưa phải full GRPO estimator**: finite-group baseline/normalization có thể đổi scaling/bias so với independent state baseline. Không dùng ví dụ này như theorem unbiasedness.

1. **Dữ liệu:** Sample prompts và nhiều completions; tính reward từ preferences/model/verifier; ghi log-probs và boundary.
2. **Học:** Tính advantages theo variant; actor update có ratio và regularization được quy định; critic/reward model được học riêng khi cần.
3. **Dùng và đánh giá:** Đánh giá trên prompts/task splits riêng, cùng decoding và token budget; đo correctness, generalization và cost.

5. So sánh, giới hạn và ứng dụng

LLM có pretrained knowledge không có nghĩa nó là RL-trained agent. Outcome reward không giám sát từng reasoning step; dài hơn có thể tăng chances cũng như cost. Pass@1 và best-of-many dùng ngân sách khác nhau, không so ngang nếu compute không khớp. Phần này cung cấp background để đọc nghiên cứu, không đòi hỏi train LLM lớn.

6. Tự luyện

1. Token là action thì state phải chứa gì?
2. Reference model và old policy có cùng vai trò không?
3. Tất cả completions cùng score: centered group rewards là gì; nó nói gì về tín hiệu học trong batch?

Đáp án: phụ lục tự luyện.

7. Điều cần nhớ và đọc thêm

RL với LLM đổi miền actions/data, không đổi nguyên tắc phân biệt objective–estimator–metric. Đọc Berkeley 14, liên hệ BC ở Mục 13.

18 Lý thuyết, giới hạn và cách đọc thực nghiệm RL

Nền tảng: Sutton–Barto Ch. 3–4, 9–11. Mở rộng: Berkeley 20, 25.

> **Mục tiêu học**
> Nhận diện assumptions của theorem; phân biệt approximation, optimization và statistical error; đọc kết quả với đúng budget.

1. Hiểu nhanh

Một theorem cho tabular MDP trả lời một câu hỏi khác với learning curve của neural agent. Lý thuyết cần xác định tập policies, dữ liệu, dynamics, estimator và chế độ giới hạn. Thực nghiệm cần xác định task, budget, evaluation distribution và variability giữa runs. Chứng minh hội tụ không cho biết một budget ngắn đã đủ học.[3, Lecture 20]

2. Bài toán được giải quyết

Input: formulation/algorithm và claim muốn kiểm tra. Output: điều kiện chính xác, kết quả được bảo đảm hoặc empirical protocol hỗ trợ claim. Prediction theorem, control theorem, sample-complexity bound và empirical improvement không thể thay cho nhau.

3. Ý tưởng và công thức cốt lõi

Bellman contraction. Với finite discounted MDP, $\mathcal{T}_\pi$ là expectation operator của Eq. (9):

$$\|\mathcal{T}_\pi V - \mathcal{T}_\pi W\|_\infty \le \gamma \|V - W\|_\infty. \tag{45}$$

$\|V - W\|_\infty$ là độ lệch lớn nhất giữa hai bảng value ở bất kỳ state nào. $\mathcal{T}_\pi$ thực hiện một exact Bellman expectation backup với cùng policy/model. Bất đẳng thức nói sau backup, độ lệch lớn nhất không vượt γ lần trước đó. Với $\gamma < 1$, repeated backups kéo estimates về cùng fixed point. Đây là kết quả cho **value vectors và exact operator**; không tự áp dụng cho optimizer cập nhật neural parameters.

Vì sao có bound này? Hai backups dùng cùng reward nên reward triệt tiêu khi lấy hiệu. Transition probabilities lấy trung bình các độ lệch, nên không vượt độ lệch lớn nhất; discount nhân thêm γ. Bellman optimality operator cũng contraction trong setting này. Projection/function approximation không tự giữ kết quả cho mọi distribution/norm.

Ba nguồn sai. Approximation error: model/value/policy class không biểu diễn được solution. Statistical error: data hữu hạn/noisy/thiếu support. Optimization error: chưa giải đủ hoặc optimizer tìm điểm kém trong class. Một run thất bại có thể có nhiều nguyên nhân, nên loss giảm chỉ loại trừ được một phần khả năng.

Discounted và average-reward. Continuing tasks có thể quan tâm reward trung bình dài hạn và differential value thay cho G_t discounted. Sutton–Barto §§10.3–10.4, 13.6 trình bày objective đó. Không bỏ γ trong một công thức discounted rồi gọi ngay là average-reward RL; Bellman/gradient và assumptions cũng đổi.

4. Ví dụ và pipeline

Với $\gamma = 0.9$ và hai vectors cách nhau max norm 2, một exact Bellman backup có distance tối đa 1.8. Nếu sau backup còn error approximation 0.5, không thể dùng số 1.8 như bảo đảm cuối cùng cho neural learner; phải phân tích error bổ sung.

1. **Dữ liệu:** Đặc tả task, dataset/interactions, policy class và seeds/splits.
2. **Học:** Đọc update cùng assumptions; kiểm tra target, support, terminal handling và train/evaluation information.
3. **Dùng và đánh giá:** Đo task metrics trên held-out evaluation; trình bày variability và compute tương ứng claim.

5. So sánh, giới hạn và ứng dụng

Quan sát	Điều có thể kiểm tra tiếp; chưa phải kết luận tự động
Critic loss giảm, return đứng	Coverage, target bias, action extraction, behavior distribution và reward design.
Model loss thấp, planner thất bại	Decision-relevant error, long rollout drift, OOD action sequences và optimizer exploitation.
Một seed đạt return cao	Variability qua independent training runs; đừng chọn seed đẹp để thay aggregate performance.
Policy thắng với ít interactions	Báo thêm gradient steps, model rollouts, wall-clock, tuning và deployment cost.
Metric ở training tasks tốt	Test distribution/tasks/goals đã giữ riêng; generalization claim cần đúng protocol.

Hãy dùng bảng để đặt **câu hỏi chẩn đoán**, không để kết luận ngay nguyên nhân. Chẳng hạn loss giảm mà return đứng có thể do target sai hoặc policy chọn action chưa tốt; hãy kiểm tra từng khâu. Một run tốt cũng chưa đo variability giữa runs. Các kiểm tra nhỏ giúp phát hiện lỗi logic, còn claim về performance/generalization cần protocol và ngân sách phù hợp.

Kiểm tra nhỏ trước nghiên cứu. Một MDP tính được reference value/policy, một example reward trễ, một case không đủ support và một test terminal boundary giúp kiểm tra logic. Những kiểm tra này không thay full benchmark/seed evaluation để claim cải thiện thuật toán. Không có phần trăm training budget cố định đủ cho mọi task.

6. Tự luyện

1. $\gamma = 0.8$, hai value vectors cách nhau 5: bound sau exact backup là gì?
2. Theorem cần infinite visitation có chứng minh dataset hữu hạn đủ không?
3. Vì sao comparison chỉ theo environment steps có thể bỏ qua chi phí model-based methods?

Đáp án: phụ lục tự luyện.

7. Điều cần nhớ và đọc thêm

Claim phải khớp setting, evidence và budget. Ch. 9–11 Sutton–Barto giúp đọc approximation/off-policy assumptions; Berkeley 20/25 giúp nối tới hạn chế hiện đại.

VI. Nền tối ưu hóa bổ trợ

19 Bổ trợ: những gì cần biết từ Convex Optimization

Nguồn phụ: Boyd–Vandenberghe Ch. 2–3, §§4.1–4.2, 4.7, Ch. 5 và §§9.1–9.3.

> **Mục tiêu học**
> Nhận diện convexity và constraint; hiểu gradient, duality/KKT; biết điều gì không thể suy ra cho deep RL.

1. Hiểu nhanh

Tối ưu hóa cung cấp ngôn ngữ để đọc objective, feasible set, gradient và constraints. Convex structure cho phép một số bảo đảm mạnh, nhưng neural policy/value objectives nhìn chung không convex. Vì vậy mục này cung cấp công cụ đọc formulation và local update, không tuyên bố mọi RL là convex optimization.[4, Ch. 1]

2. Bài toán được giải quyết

Boyd dùng decision variable x, objective $f_0(x)$, constraints $f_i(x) \le 0$ và $h_j(x) = 0$. Output là feasible solution/optimum dưới assumptions. Trong RL, policy probabilities/occupancies và neural parameters là các variables khác nhau; convexity phải kiểm tra theo đúng variable.

3. Ý tưởng và công thức cốt lõi

Convexity. Convex set chứa đoạn nối hai điểm bất kỳ của nó. Convex function có domain convex và thỏa:

$$f(\tau x + (1-\tau)y) \le \tau f(x) + (1-\tau) f(y), \qquad 0 \le \tau \le 1. \tag{46}$$

$\tau x + (1-\tau)y$ là một điểm trên đoạn nối x và y. Vế phải nội suy hai values ở hai đầu; convexity đòi value ở điểm giữa không cao hơn mức nội suy đó. Với $\tau = 1/2$, ta kiểm tra midpoint và average hai endpoint values. Nhưng **kiểm tra một cặp điểm có chứng minh convex không?** Không: điều kiện phải đúng với mọi cặp trong convex domain và mọi $\tau \in [0,1]$. τ ở đây chỉ là mixing coefficient.

Convex problem cần objective/inequality functions convex và equalities affine. Local minimum là global minimum; strict convexity có thể cho uniqueness nếu optimum tồn tại. Convexity không tự bảo đảm tồn tại nghiệm hoặc data đúng.[4, §4.2]

Gradient và curvature. Gradient hướng tăng nhanh nhất theo local Euclidean approximation; descent đi ngược, ascent đi cùng. Step size quyết định có giảm/tăng objective thật không, nên không nói gradient step bất kỳ bảo đảm improvement.

$$x_{k+1} = x_k - \alpha_k \nabla f_0(x_k). \tag{47}$$

Gradient chỉ hướng tăng nhanh nhất cục bộ theo Euclidean geometry. Vì muốn minimize f_0, đi ngược gradient một bước dài theo α_k. k đếm vòng tối ưu; step size quá lớn có thể vượt qua vùng giảm và làm objective tăng. Trong actor ascent, muốn maximize performance nên đổi sang dấu cộng. Dấu đúng chỉ xác định hướng cục bộ, chưa bảo đảm một bước hữu hạn hoặc stochastic estimate sẽ cải thiện objective thật.

Hessian mô tả curvature; với twice-differentiable function trên open convex domain, Hessian positive semidefinite ở mọi điểm tương đương convexity. Kiểm tra một điểm/mini-batch không chứng minh global convexity. Natural gradient thay geometry; trust region dùng local model dưới constraint.

Lagrangian và duality.

$$L_{\text{Lag}}(x, \lambda, \nu) = f_0(x) + \sum_i \lambda_i f_i(x) + \sum_j \nu_j h_j(x). \tag{48}$$

Bắt đầu từ objective $f_0(x)$, rồi thêm mỗi constraint nhân với một multiplier. Với $f_i(x) \le 0$, dùng $\lambda_i \ge 0$; equality $h_j(x) = 0$ dùng ν_j không bị hạn chế dấu. Khi minimize theo x, công thức gom objective và ảnh hưởng của constraints vào một biểu thức; dual còn tối ưu các multipliers để tìm lower bound tốt. Multipliers có vai trò toán học của constrained problem, không phải penalty tùy ý bảo đảm feasibility. L_{Lag} khác training loss L.

Dual function lấy infimum Lagrangian theo x; dual maximize bound theo multipliers. Weak duality cho lower bound cả khi primal không convex. Strong duality cần điều kiện, ví dụ Slater trong convex setting; thêm multiplier vào neural loss không đủ.[4, §§5.1–5.3]

KKT. Gồm primal feasibility, dual feasibility, stationarity và complementary slackness. Với differentiable convex problem, KKT đủ cho optimum; necessity/attainment cần assumptions phù hợp, ví dụ constraint qualification.

$$\nabla_x L_{\text{Lag}}(x^*, \lambda^*, \nu^*) = 0, \qquad \lambda_i^* f_i(x^*) = 0 \quad \text{cho mọi } i. \tag{49}$$

Stationarity yêu cầu các gradients của objective và active constraints cân bằng qua multipliers tại nghiệm ứng viên. Complementary slackness nói $\lambda_i^* f_i(x^*) = 0$: constraint còn dư strict slack thì multiplier bằng zero; multiplier dương thì constraint phải active. Constraint active vẫn có thể có multiplier zero. Còn phải kiểm tra $f_i(x^*) \le 0$, $h_j(x^*) = 0$ và $\lambda_i^* \ge 0$. Với differentiable convex problem, bộ KKT đủ cho optimum; necessity cần assumptions phù hợp.

Vector optimization. Chọn thứ tự của objective vector, thường componentwise, rồi nói Pareto optimality. Weighted sum chọn một trade-off; trong nonconvex attainable set, nó có thể bỏ qua unsupported Pareto points. Boyd §4.7 không tự cung cấp MORL policy solution concepts, SER/ESR hoặc protocol hypervolume; những phần đó cần nguồn chuyên biệt. "Một scalarized policy" không đồng nghĩa "toàn bộ Pareto front".

4. Ví dụ và pipeline

Minimize $f_0(x) = (x-2)^2$ với $x \ge 3$, tức $f_1(x) = 3 - x \le 0$. Optimum $x^* = 3$. Gradient objective tại 3 là 2, gradient constraint là -1; stationarity cần $\lambda_1^* = 2$. Complementary slackness đúng vì constraint active. Nếu dùng penalty coefficient tùy ý thay cho optimum multiplier, kết quả không tự thỏa constraint.

1. **Dữ liệu:** Xác định variable, objective, feasible set và source of estimates.
2. **Học:** Kiểm tra convexity/regularity; chọn gradient/solver/dual update theo đúng setting.

3. **Dùng và đánh giá:** Kiểm tra feasibility, optimality conditions hoặc empirical performance riêng; không chỉ báo penalized loss.

5. So sánh, giới hạn và ứng dụng

Unconstrained loss training, constrained optimization và Lagrangian relaxation khác nhau ở feasible set và certificate. Convex optimization hỗ trợ hiểu policy trust region, constrained RL và vector trade-offs; deep RL vẫn có nonconvexity, distribution shift và noisy estimators. Không cần học toàn bộ interior-point implementation để đọc pipeline RL cơ bản.

6. Tự luyện

1. Với ví dụ nhưng constraint $x \geq 1$, optimum và multiplier là gì?
2. Convex squared error theo linear weights có làm return objective theo neural policy convex không?
3. Weighted sum có luôn thu được mọi Pareto solution trong nonconvex set không?

Đáp án: phụ lục tự luyện.

7. Điều cần nhớ và đọc thêm

Giữ convexity, gradient, constraints, duality/KKT và vector order; đọc Boyd đúng các phần nêu trên. Chỉ áp guarantees khi giả thiết khớp formulation RL.

VII. Tự luyện và tra cứu

Phụ lục A. Đáp án tự luyện

> **Cách tự kiểm tra**
> Làm câu hỏi trước khi đọc đáp án. Các số dưới đây là kết quả của ví dụ tự dựng; lời giải yêu cầu cả số đúng và lý do đúng. Nếu chỉ nhớ công thức mà không xác định terminal, policy hoặc data support thì chưa đạt mục tiêu học.

1. RL, MDP và return

Trở về câu hỏi.

(1) $G_0 = 2 - 0.5 + 0.25 \cdot 3 = 2.25$, $G_1 = -1 + 0.5 \cdot 3 = 0.5$. (2) Cùng vị trí có thể khác vận tốc hoặc trạng thái ẩn; hậu quả action khác dù ảnh hiện tại giống nhau. (3) Không: computational cutoff có thể chỉ là truncation. Nếu horizon limit là một phần task, thời gian còn lại cũng có thể cần đưa vào state.

2. Value và Bellman

Trở về câu hỏi.

(1) $q_\pi(s, a) = 2 + 0.9 \cdot 0.75 \cdot 2 = 3.35$; policy đều có $v_\pi(s) = (3.35 + 3)/2 = 3.175$. (2) Greedy thay action probabilities, nên future trajectories và value cũng đổi. (3) Không nói chung; cần model/reward để lookahead hoặc một action-value/actor phù hợp.

3. Dynamic programming

Trở về câu hỏi.

(1) Action a cho $0.4 \cdot 4 = 1.6$, b cho 2, nên chọn b. (2) Có: in-place sweep cập nhật $u = 4$ rồi dùng ngay cho s, cho 3.6 trong một sweep; synchronous dùng $V_0(u) = 0$. (3) Không: contraction của model sai chỉ giải fixed point của model đó.

4. MC và TD

Trở về câu hỏi.

(1) Target $1 + 0.8 \cdot 5 = 5$; $\delta = 5 - 2 = 3$; $V(s)$ mới 2.3. (2) Target chỉ còn R_{t+1} vì terminal value zero. (3) MC dùng realized rewards của phần còn lại; TD dùng learned next-state estimate, nên chúng không nhất thiết trùng.

5. Sarsa và Q-learning

Trở về câu hỏi.

(1) Sarsa target 4.6, estimate 3.3; Expected Sarsa target 2.8, estimate 2.4; Q-learning target 4.6, estimate 3.3. (2) Không: fixed ϵ không greedy in the limit; infinite visitation cũng cần chứng minh riêng. (3) Không: theorem yêu cầu coverage, không áp cho pair chưa được học.

6. Multi-step, traces và off-policy

Trở về câu hỏi.

(1) Không bootstrap: $1 + 0.9 \cdot 2 = 2.8$. (2) $0.8 \cdot 0.64 + 1 = 1.512$. (3) Không: support condition bị vi phạm, ratio không xác định; không có samples để đánh giá action đó.

7. Function approximation và DQN

Trở về câu hỏi.

(1) $y = 2 + 0.9 \cdot 5 = 6.5$ khi nonterminal; $y = 2$ khi terminal. (2) Prediction 2, error -1, nên $w = (-0.1, 0.8)^\top$. (3) Shared nonlinear parameters, distribution và bootstrap/off-policy làm assumptions và update geometry khác tabular.

8. Policy gradient và actor–critic

Trở về câu hỏi.

(1) Advantage sample -1, nên update đi ngược log-gradient của action đã sample, giảm probability ở local step nhỏ. (2) $1 + 0.8 \cdot 2 = 2.6$. (3) Action-dependent baseline có thể có expected score contribution khác zero; muốn dùng cần correction/derivation phù hợp.

9. PPO và maximum-entropy RL

Trở về câu hỏi.

(1) Unclipped term -1.4, clipped term -1.6, minimum -1.6. (2) Không: clipped surrogate không enforce pointwise/global KL constraint. (3) Có: entropy thay preference trong objective, nên có thể đổi optimum.

10. Exploration

Trở về câu hỏi.

(1) Random part không xét uncertainty, nên thử cả action đã biết kém. (2) Policy có thể bị thu hút bởi randomness không học được thay vì task progress. (3) Bandit thử arm trực tiếp; navigation cần tới đúng state qua chuỗi action trước khi thấy reward.

11. Planning và model-based RL

Trở về câu hỏi.

(1) Không tăng real interaction count; vẫn tăng model compute. (2) Training distribution khác planned trajectory distribution; model error ở state/action quan trọng hoặc rollout dài có thể bị optimizer khai thác. (3) Feedback nằm ở observation thật sau action đầu và việc lập kế hoạch lại.

12. World models

Trở về câu hỏi.

(1) Không: chi tiết nhỏ có thể quyết định reward nhưng đóng góp ít vào reconstruction error. (2) Không: actor có thể chọn action trực tiếp từ latent estimate; planning là lựa chọn khác. (3) Future observations không có ở deployment; cần causal estimator/filtering tương ứng và tránh information leakage.

13. Imitation và inverse RL

Trở về câu hỏi.

(1) Không mâu thuẫn: validation trên expert states, execution trên learner-induced states. (2) Không nguyên bản; yêu cầu expert labels ở states learner thăm phải được đáp ứng hoặc đổi setting/method. (3) Không: nhiều reward hypotheses có thể cùng giải thích demonstrations.

14. Offline RL

Trở về câu hỏi.

(1) Không: size không thay action/state support. (2) Không, nếu nó vẫn thu interactions mới khi học thì là online off-policy. (3) Weight 1; một realization không mô tả variance, coverage hoặc bias của estimator.

15. Inference

Trở về câu hỏi.

(1) ELBO $= -3 - 1.2 = -4.2$. (2) Đổi hai arguments đổi expectation distribution và log ratio; giá trị nói chung khác. (3) Không, trừ khi quyền điều khiển dynamics là một phần problem thật; naive inference có thể giải một optimistic problem khác.

16. Multi-task và hierarchy

Trở về câu hỏi.

(1) $q_\pi = 3 - 2 = 1$. (2) Không: successor features đánh giá policy đang có cho reward mới; optimal control còn cần improvement/coverage. (3) Transition-reward labels không còn đúng task, làm target critic sai.

17. LLM RL

Trở về câu hỏi.

(1) Prompt và prefix đã sinh, cộng information/task state cần thiết. (2) Không: old policy là sampling policy cho ratio; reference là distribution để regularize. (3) Centered rewards bằng zero, batch không có relative outcome signal dưới quy tắc ấy; regularizers hoặc tín hiệu khác vẫn có thể tạo update.

18. Theory và evaluation

Trở về câu hỏi.

(1) Max-norm bound $0.8 \cdot 5 = 4$. (2) Không: asymptotic visitation không định lượng finite-sample sufficiency. (3) Model rollouts, gradient steps và search có cost riêng; interaction efficiency không bằng total computational efficiency.

19. Convex Optimization

Trở về câu hỏi.

(1) $x^* = 2$, constraint inactive và $\lambda_1^* = 0$. (2) Không: đây là objective khác, variable khác và có thể distribution phụ thuộc policy. (3) Không: unsupported Pareto points có thể không là optimum của bất kỳ positive weighted sum nào.

Phụ lục B. Bản đồ nguồn để kiểm tra và học tiếp

Cách truy nguồn. Các công thức trong phần chính đi kèm giải thích cách đọc và sử dụng. Muốn đối chiếu định nghĩa hoặc assumptions, mở lecture/chapter tương ứng trong bảng dưới đây; ký hiệu chung đã được định nghĩa ở đầu tài liệu. Các links Berkeley trỏ tới slides của course Spring 2026.

Mục	Từ khóa	Nguồn nên mở
1	RL, MDP, return	Silver 1–2; Sutton–Barto §§3.1–3.4.
2	Value, Bellman	Silver 2; Sutton–Barto §§3.5–3.7.
3	DP, GPI	Silver 3; Sutton–Barto §§4.1–4.6.
4	MC, TD prediction	Silver 4; Sutton–Barto §§5.1–5.2, 6.1–6.3.
5	Sarsa, Q-learning	Silver 5; Sutton–Barto §§6.4–6.7.
6	n-step, traces, IS	Silver 4–5; Sutton–Barto Ch. 7, §§12.1–12.5, §5.5.
7	Approximation, DQN	Silver 6; Sutton–Barto Ch. 9–11; Berkeley 2026, Lecture 7: Value-Based RL, Berkeley 2026, Lecture 8: Q-learning in Practice.
8	PG, actor–critic	Silver 7; Sutton–Barto Ch. 13; Berkeley 2026, Lecture 5: Policy Gradients, Berkeley 2026, Lecture 6: Actor Critic.
9	PPO, TRPO, SAC	Berkeley 2026, Lecture 9: Advanced PG Part 1, Berkeley 2026, Lecture 10: Advanced PG Part 2, Berkeley 2026, Lecture 13: Control as Variational Inference.
10	Exploration	Silver 9; Sutton–Barto Ch. 2; Berkeley 2026, Lecture 19: Exploration, Berkeley 2026, Lecture 23: Advanced Exploration.
11	Dyna, model-based, MPC	Silver 8; Sutton–Barto Ch. 8; Berkeley 2026, Lecture 15: Model-Based RL Part 1, Berkeley 2026, Lecture 16: Model-Based RL Part 2.
12	World models, POMDP	Berkeley 2026, Lecture 16: Latent-state models, Berkeley 2026, Lecture 14: Sequences, LLMs and partial observability.
13	BC, DAgger, IRL	Berkeley 2026, Lecture 2: Behavioral Cloning, Berkeley 2026, Lecture 3: BC Part 2, Berkeley 2026, Lecture 13: IRL and adversarial imitation.
14	Offline RL	Berkeley 2026, Lecture 17: Offline RL Part 1, Berkeley 2026, Lecture 18: Offline RL Algorithms.
15	VI, control as inference	Berkeley 2026, Lecture 11: Variational Inference, Berkeley 2026, Lecture 12: VI in RL, Berkeley 2026, Lecture 13: Control as VI.
16	Multi-task, goals, options	Berkeley 2026, Lecture 24: Multi-Task and Hierarchical RL.
17	LLM RL, preferences	Berkeley 2026, Lecture 14: RL with Sequences and LLMs.
18	Theory, evaluation	Berkeley 2026, Lecture 20: RL Theory, Berkeley 2026, Lecture 25: Challenges and Open Problems.

Mục	Từ khóa	Nguồn nên mở
19	Convex Optimization	Boyd Ch. 2–3, §§4.1–4.2, 4.7, Ch. 5, §§9.1–9.3.

Cầu nối tới nghiên cứu. Khi đọc một paper mới, xác định setting, training/deployment I/O, objective, estimator/update, assumptions và evidence trước. Dẫn về mục nền tảng tương ứng: learned dynamics về Mục 11/12; offline methods về Mục 14; preference/reward learning về Mục 13/17; transfer về Mục 16. Đây là quy tắc tổ chức note, không đánh đồng các họ bài toán.

Phạm vi chưa triển khai sâu. Muốn học chuyên sâu average-reward và off-policy approximation, đọc đầy đủ Ch. 10–11, 13.6 Sutton–Barto. Safe RL, MORL và reward-free RL cần thêm bài giảng/paper chuyên biệt; bốn nguồn này cung cấp nền để đọc tiếp, không xác lập đầy đủ các problem formulations đó.

Tài liệu tham khảo

[1] David Silver. *Reinforcement Learning*, UCL, 2015. Lectures 1–9. Trang tác giả và liên kết slide: https://www.davidsilver.uk/teaching/. Các slide đã đối chiếu theo bản PDF hiện được trang tác giả dẫn tới; thời điểm truy cập 11/10/2026.

[2] Richard S. Sutton and Andrew G. Barto. *Reinforcement Learning: An Introduction*, 2nd ed., MIT Press, 2018; author PDF bản hiệu đính 2020. Ch. 2–13. Trang xuất bản: https://mitpress.mit.edu/9780262352703/reinforcement-learning/. PDF tác giả: http://incompleteideas.net/book/RLbook2020.pdf.

[3] Sergey Levine và đội ngũ CS185/285, UC Berkeley. *Deep Reinforcement Learning*, Spring 2026. Lectures 2–20, 23–25 theo bản đồ ở Phụ lục B. https://rail.eecs.berkeley.edu/deeprlcourse/. Tên chính xác trong từng PDF có thể cụ thể hơn nhãn trên course page; nội dung và công thức được đối chiếu với PDF.

[4] Stephen Boyd and Lieven Vandenberghe. *Convex Optimization*, Cambridge University Press, 2004, bản PDF tác giả. https://web.stanford.edu/~boyd/cvxbook/. Full PDF: https://web.stanford.edu/~boyd/cvxbook/bv_cvxbook.pdf. Các phần dùng: Ch. 2–3, §§4.1–4.2, 4.7, Ch. 5, §§9.1–9.3.